



Hypergraph: Modeling Higher-Order Interactions in the Real World

2026-01-15

InHyeok Jeong

School of Computer Science and Engineering
Chung-Ang University
MINDS Lab

Content

- ☐ Introduction
- ☐ Hypergraph Expansions for Message Passing Design
- ☐ Self-Supervised Hypergraph Learning



From pairwise to group-wise



□ Co-authorships of Researchers

Enhancing Hyperedge Prediction with Context-Aware Self-Supervised Learning

Yunyong Ko, Hanghang Tong, *Fellow, IEEE*, and Sang-Wook Kim*, *Senior Member, IEEE*

HNHN: Hypergraph Networks with Hyperedge Neurons

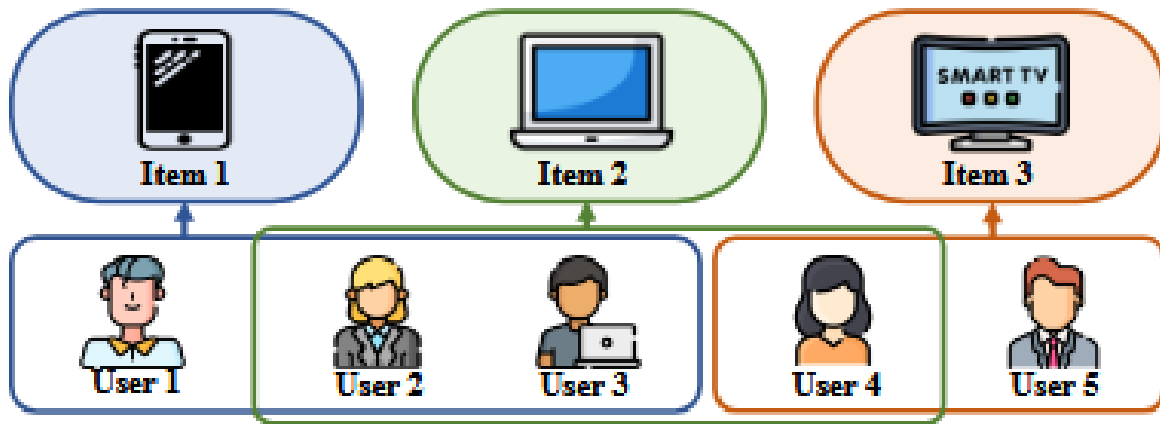
Yihe Dong
Microsoft

Will Sawin
Department of Mathematics
Columbia University

Yoshua Bengio
Mila
Université de Montréal

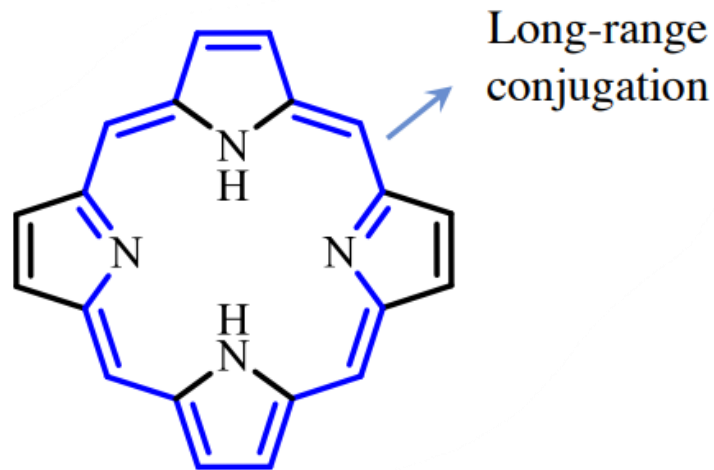
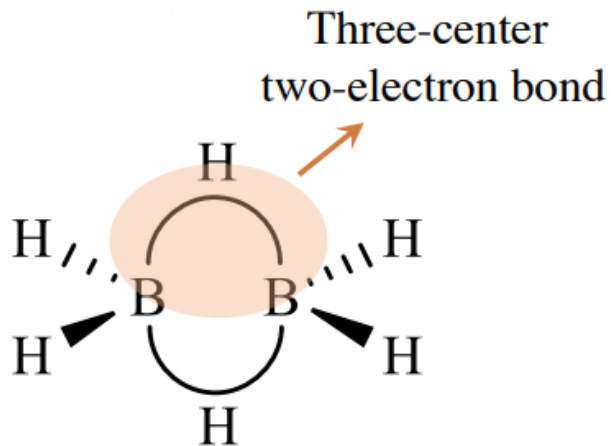
Higher-Order Interactions are Everywhere

□ Co-purchase of items



Higher-Order Interactions are Everywhere

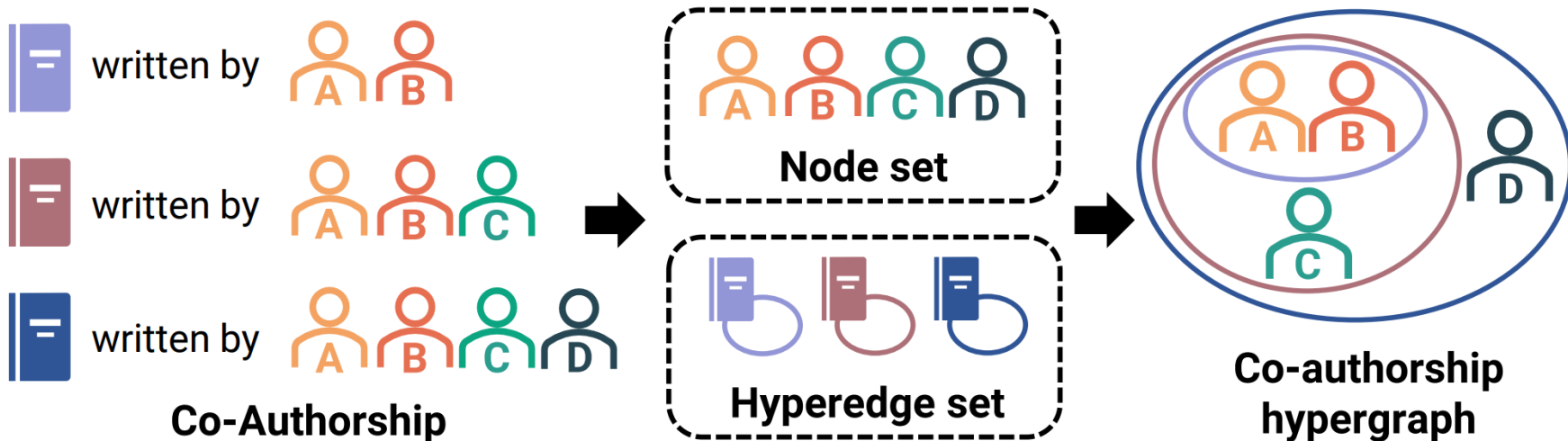
□ Molecule Structure



Hypergraphs Model Higher-Order Interactions

□ Hypergraphs can naturally model higher-order interactions as **hyperedges**

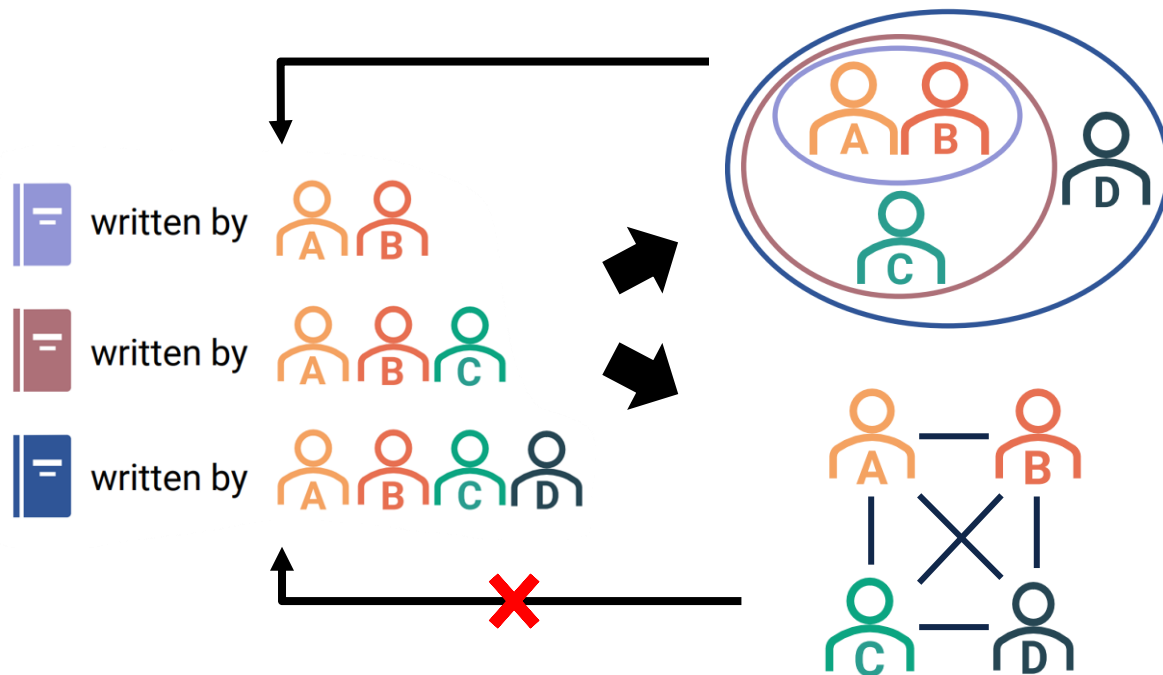
■ A **hyperedge** is subset of vertices



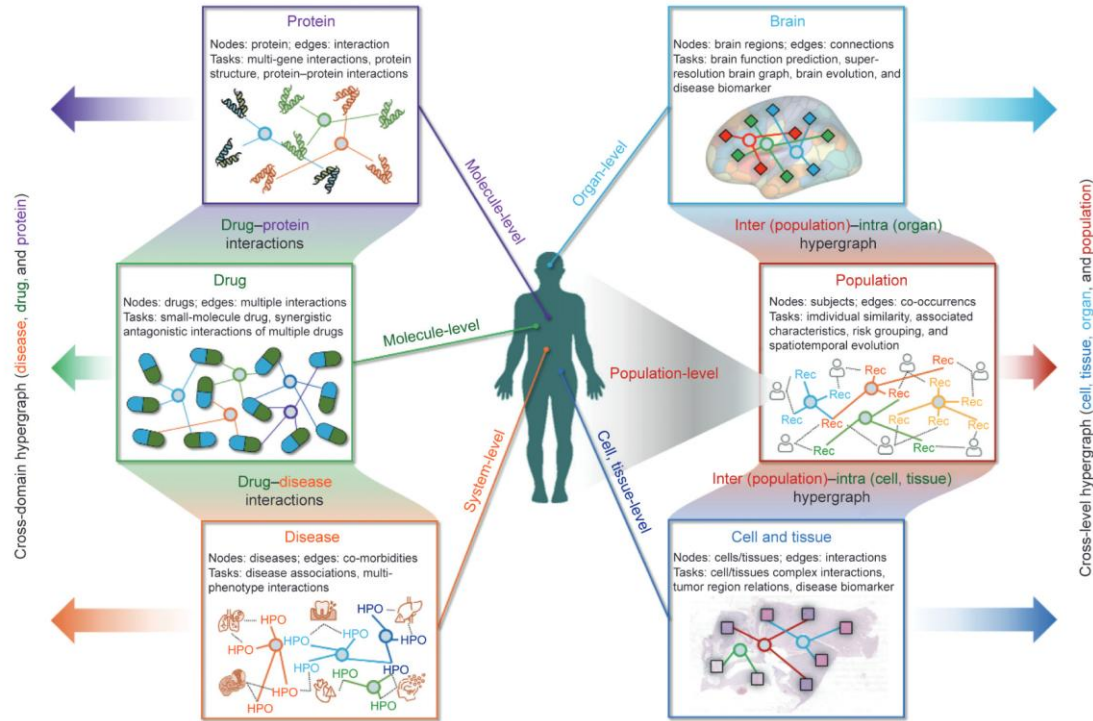
Limitation of Graphs

□ Using graphs can incur **information loss**

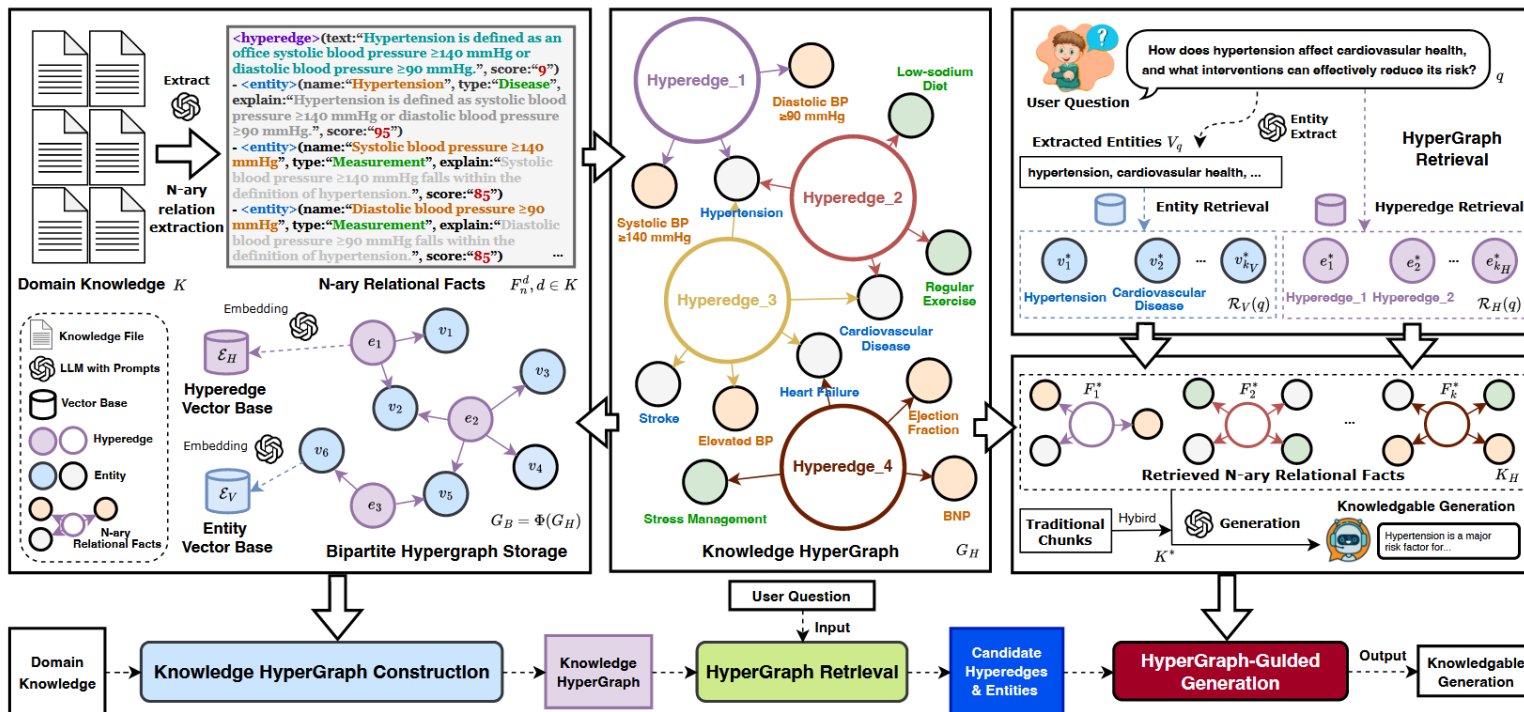
- Unable to recover higher-order interactions from simple graphs



□ Intelligent Medicine



Information Retrieval



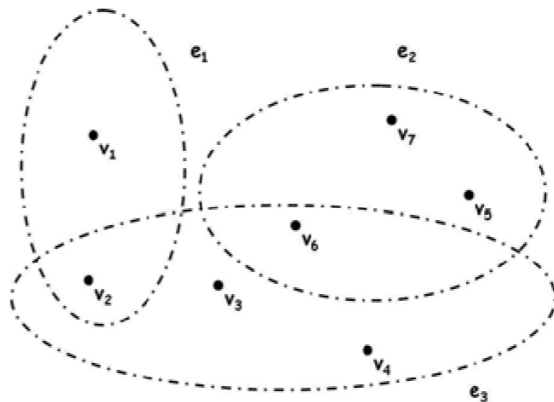


Hypergraph Expansions for Message Passing Design

Hypergraph Representation

□ A hypergraph $G = (V, E)$

- V : a finite set of objects, E : a family of subsets e of V s.t $\cup_{e \in E} e = V$
- $|V| \times |E|$ matrix H with entries $h(v, e) = 1$ if $v \in e$ and 0 otherwise



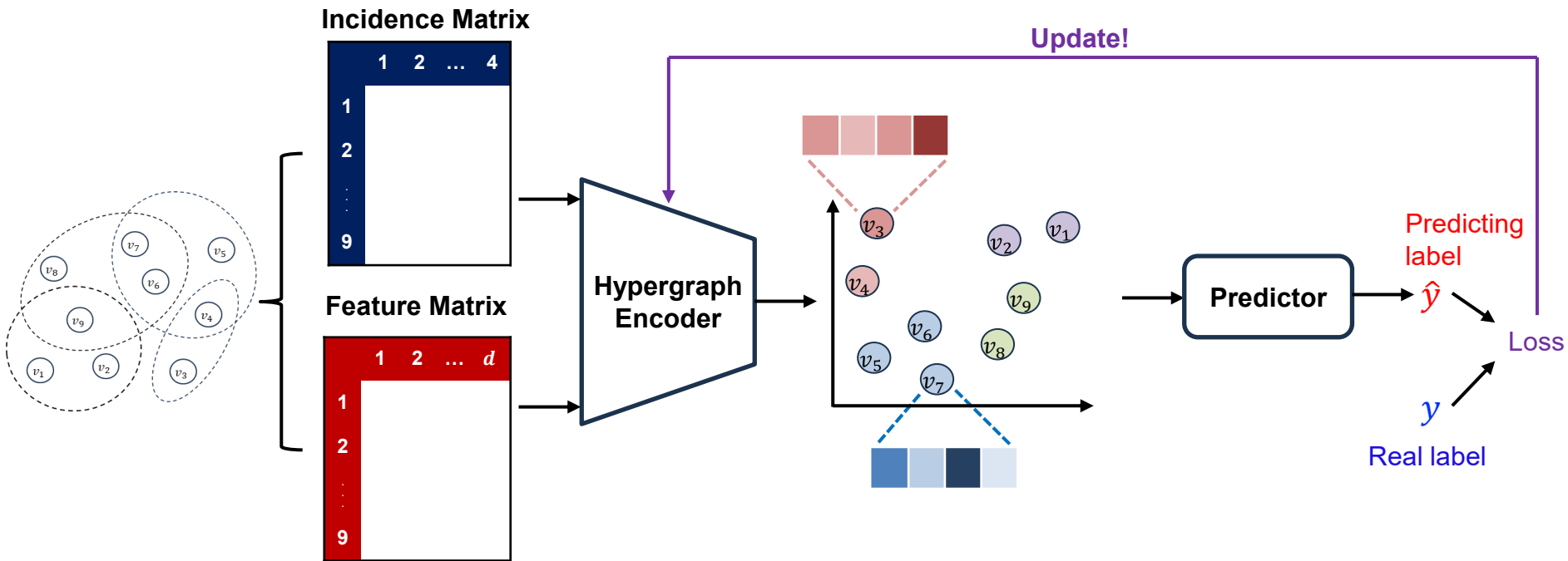
Incidence Matrix

	1	2	3
1	1	0	0
2	1	0	1
3	0	0	1
4	0	0	1
5	0	1	0
6	0	1	1
7	0	1	0

Hypergraph Neural Networks

□ Same as graph neural networks

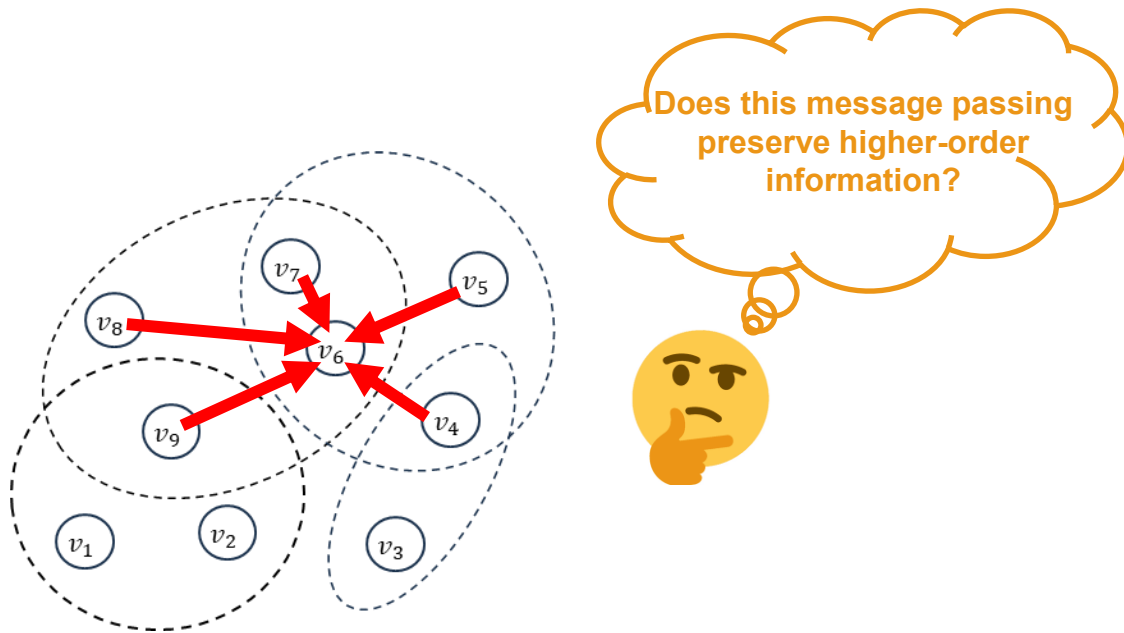
■ Learn embeddings by **aggregating** and **transforming signals** induced by **incident hyperedges**



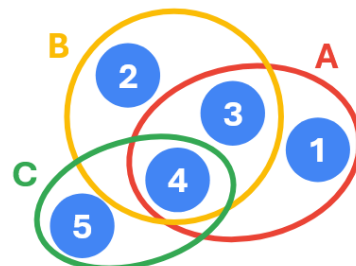
Challenges

□ How to expand hypergraphs into pairwise graphs for message passing?

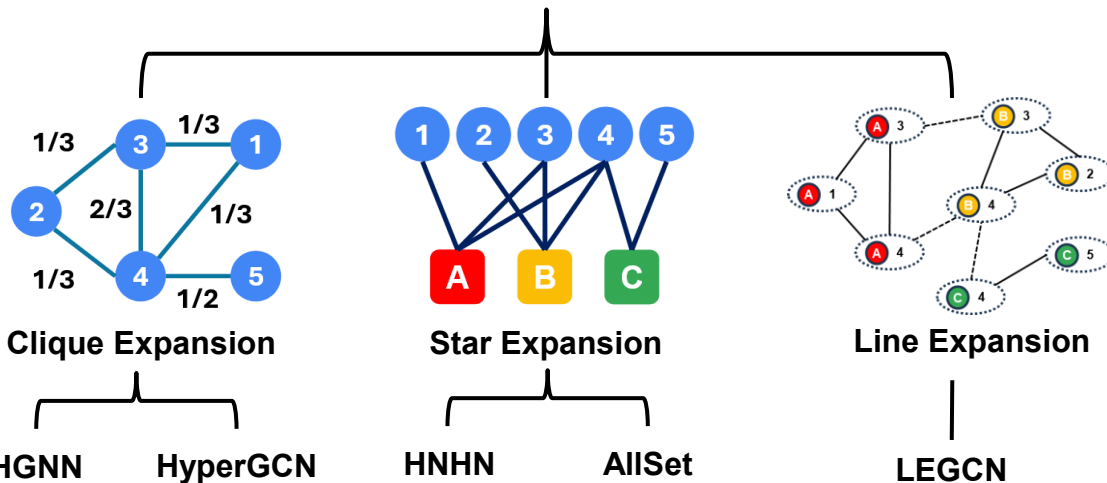
- Preserving higher-order information during the expansion is critical



Hypergraph Modeling



Input Hypergraph

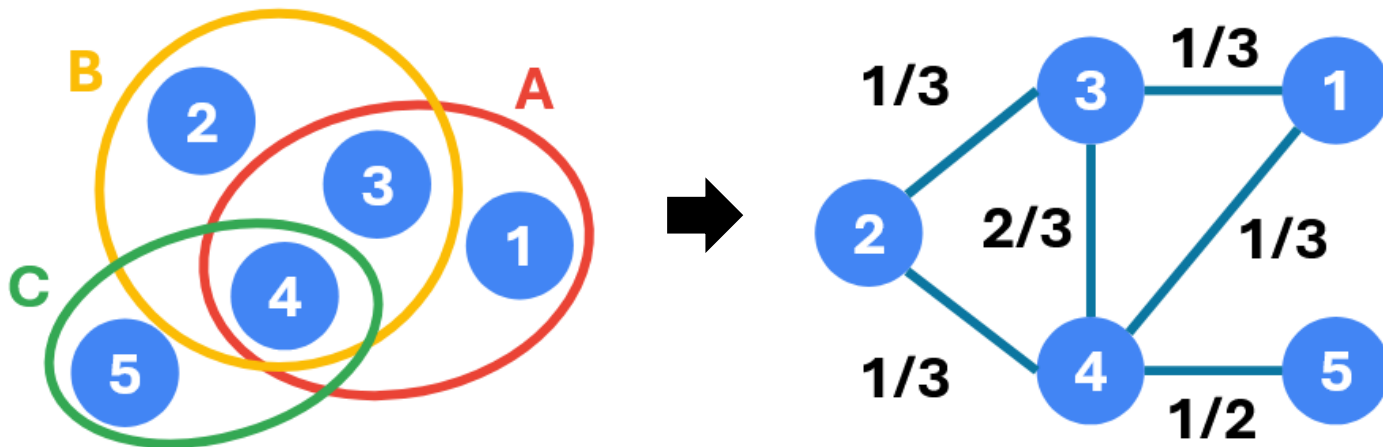


Clique Expansion

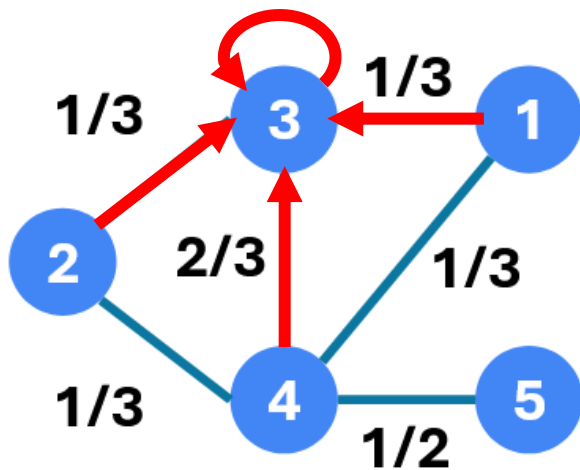
□ Replace each hyperedge with a **clique**

■ Node-to-Node

■ Assigning proper edge weights is crucial for capturing HOIs



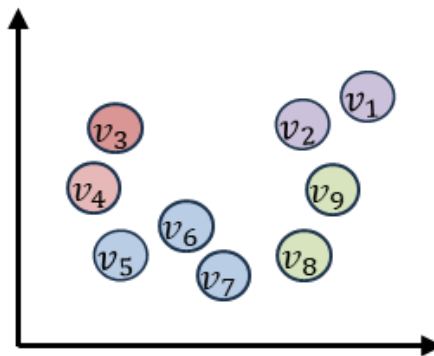
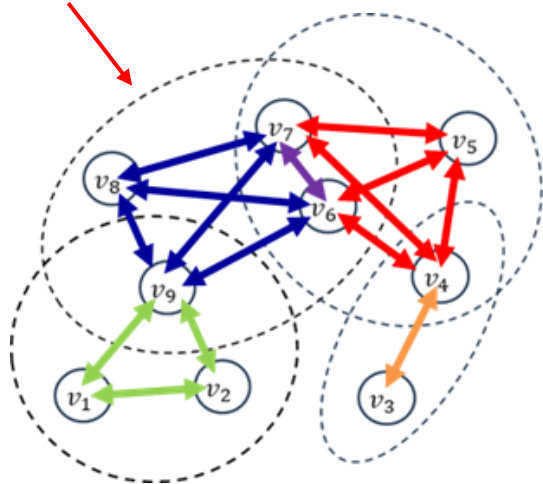
- Perform message passing **between nodes**



□ Motivation

- **Over-smoothing** & **Time complexity** problem can arise

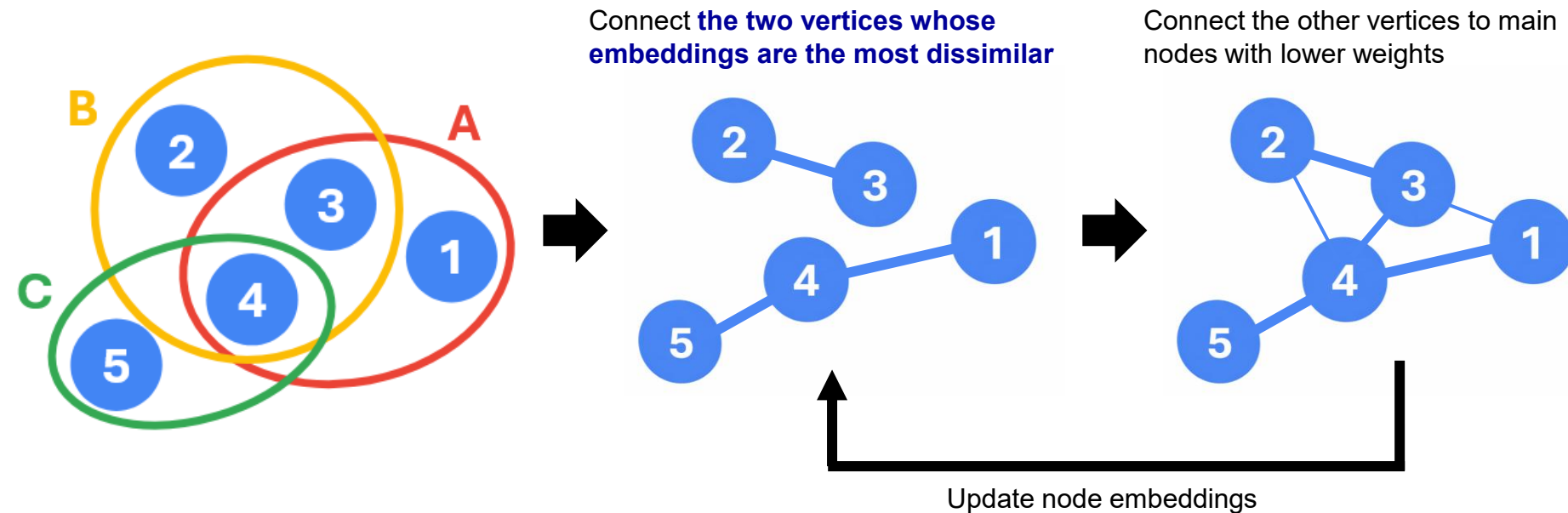
$O(|s|^2)$ per each hyperedge



As message passing progresses, all node embeddings become similar



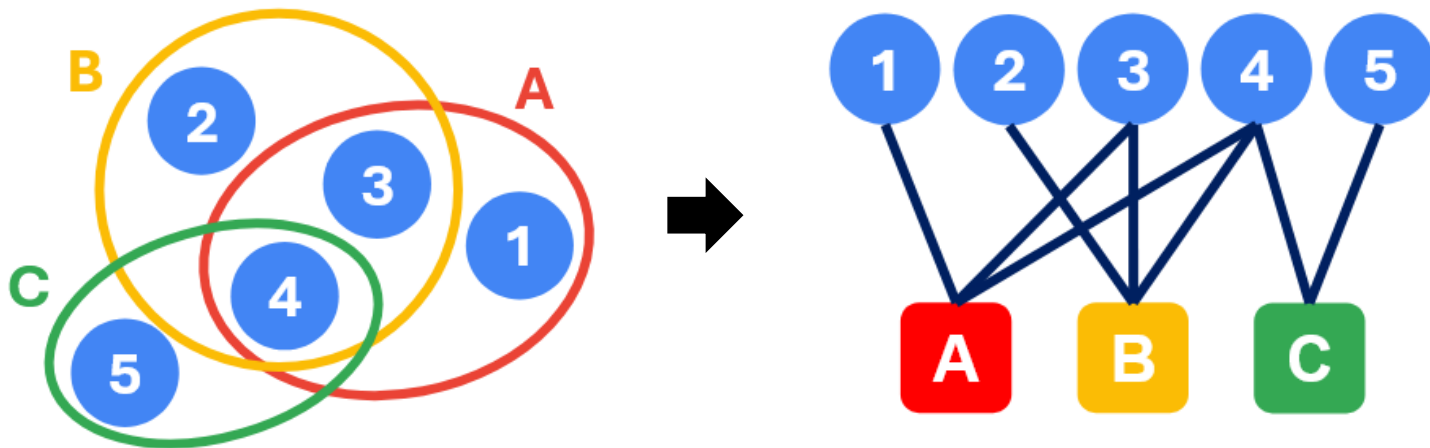
□ Advanced Clique Expansion



Star Expansion

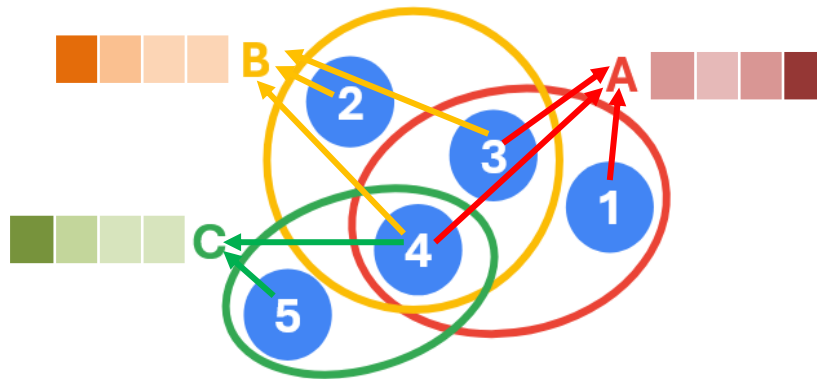
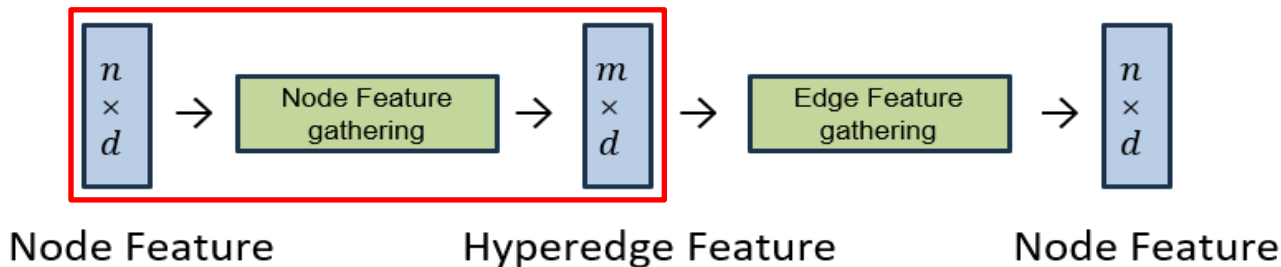
□ View both vertices and hyperedges as nodes

■ Make **bipartite graph** (vertices $\leftrightarrow$ hyperedges)



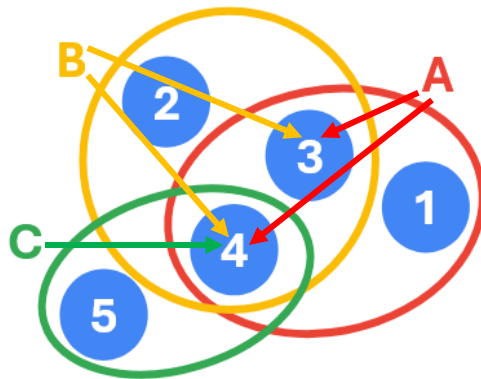
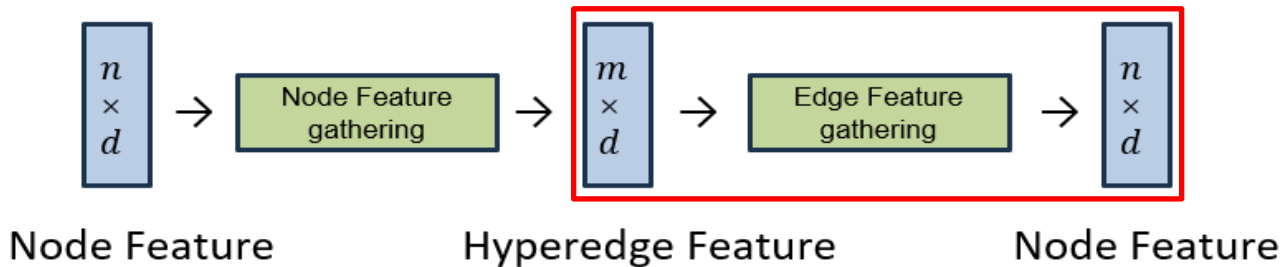
□ Define hyperedge features through nodes that it contains

■ $X'_E = \sigma(D_{E,l,\beta}^{-1} A D_{V,r,\beta} X_V W_E + b_E)$



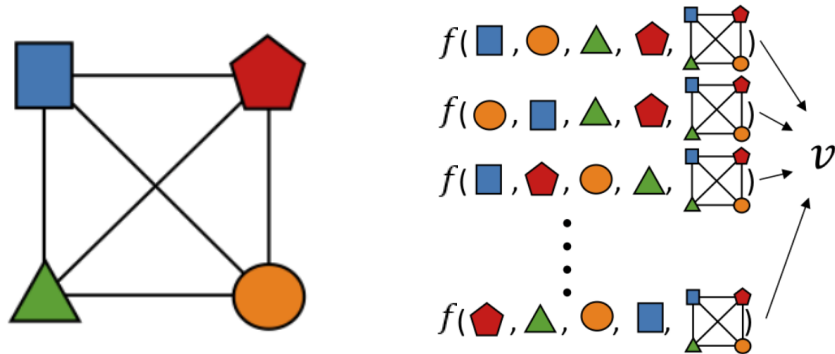
□ Define nodes through hyperedges that contain itself

■ $X'_V = \sigma(D_{V,l,\alpha}^{-1} A D_{E,r,\alpha} X'_E W_V + b_V)$



□ What is a multiset function?

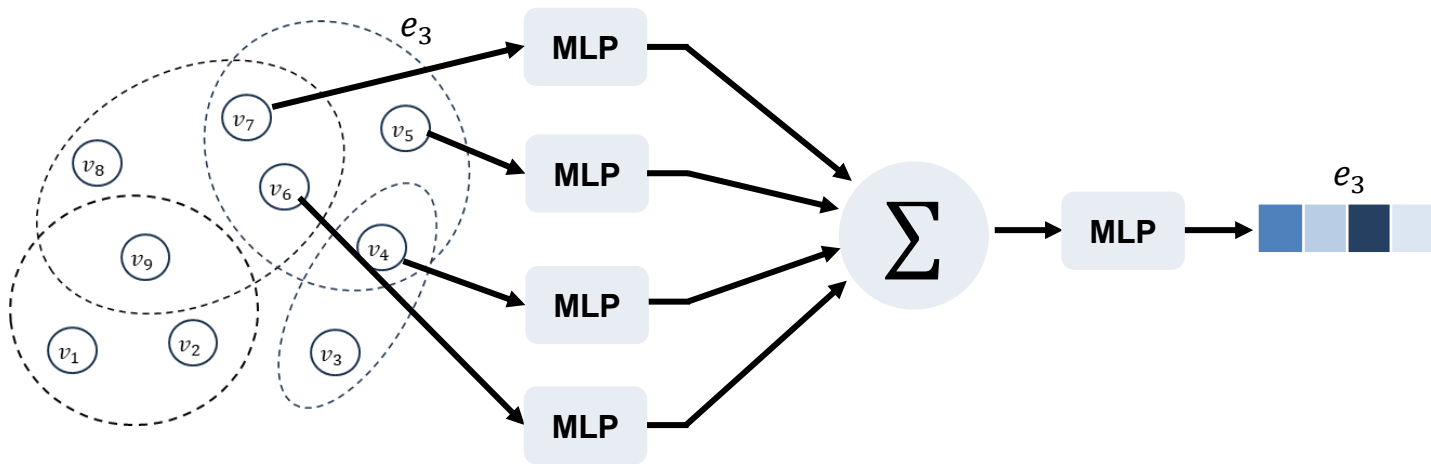
- A function f is a multiset function if it is permutation invariant
- A multiset function can formulate aggregation for message passing



□ Key Idea

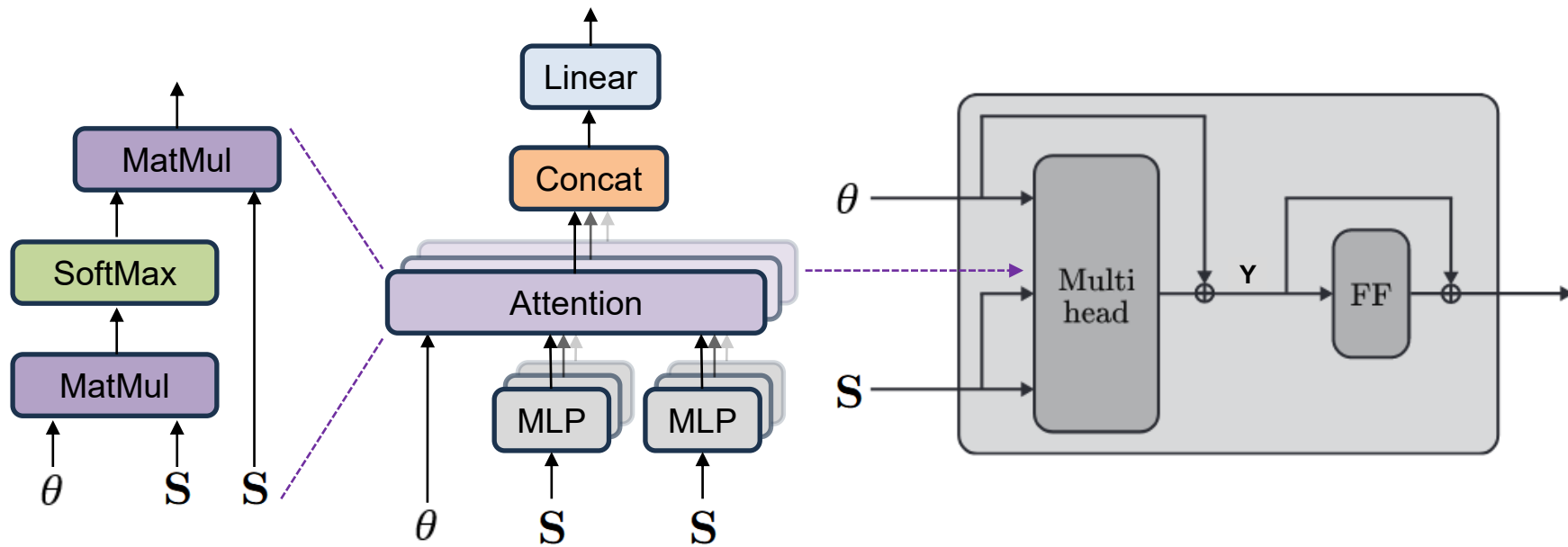
■ Any multiset functions f can be parametrized

- $f(S) = \rho(\sum_{s \in S} \phi(s))$, where ρ and ϕ are some bijective mappings
- These mappings can be replaced by any universal approximator such as a MLP



□ AllSetTransformer

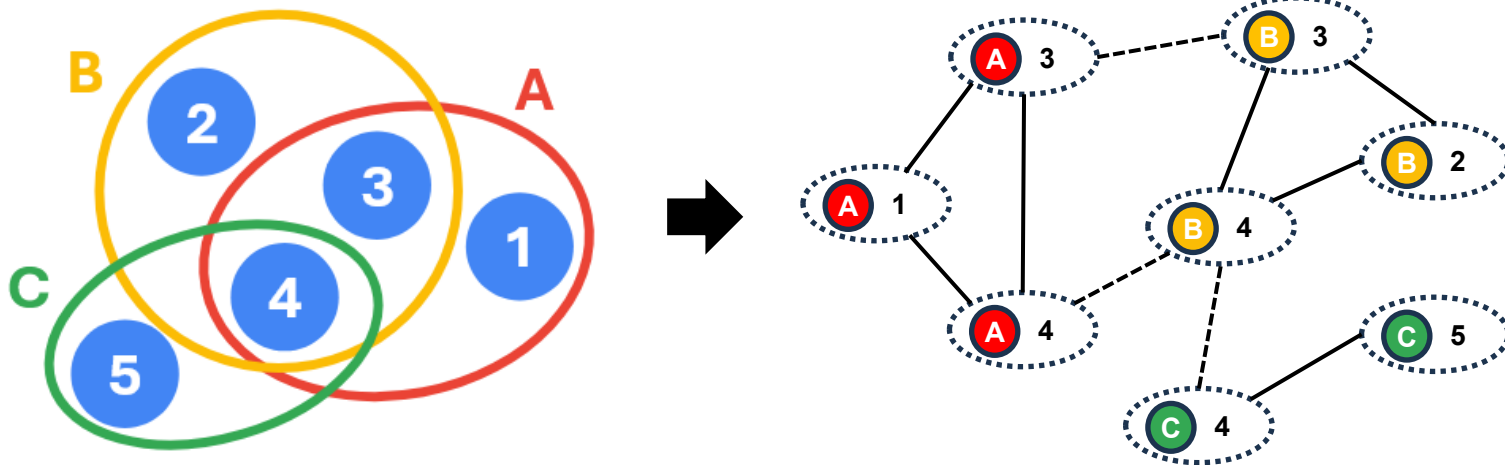
- Attention-based AllSet layer for hypergraph neural networks
- Reflect similarity between two vertices in terms of θ



Line Expansion

□ View node-hyperedge relations as nodes

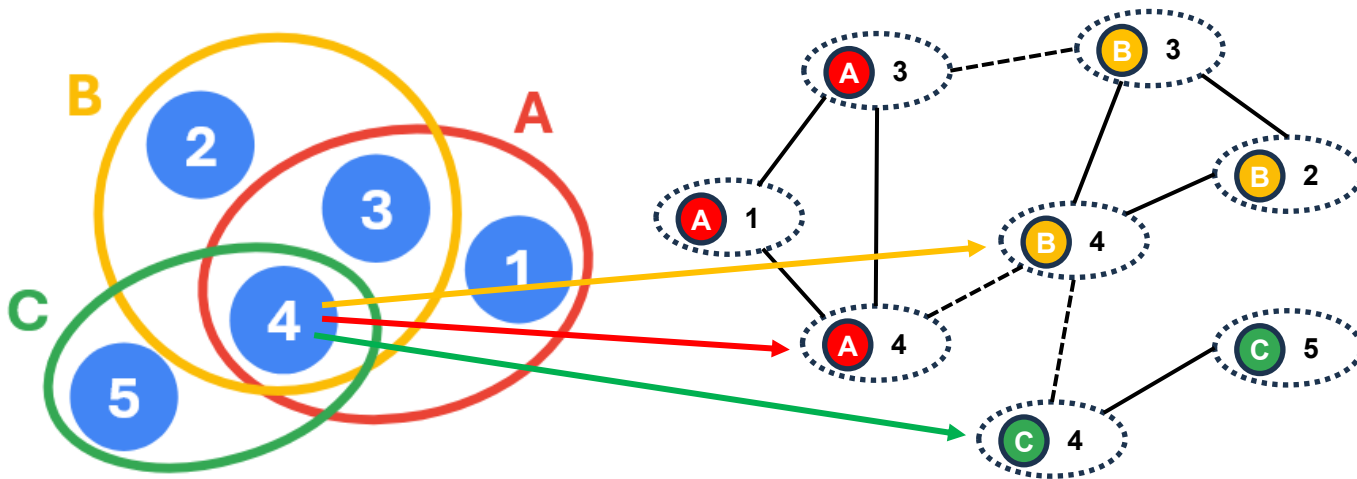
- View each node as **a vertex with hyperedge context** or **a hyperedge with vertex context**



□ Feature Projection

- Scatter features from vertex of $\mathcal{G}_H$ to feature vectors of line nodes in $\mathcal{G}_l$

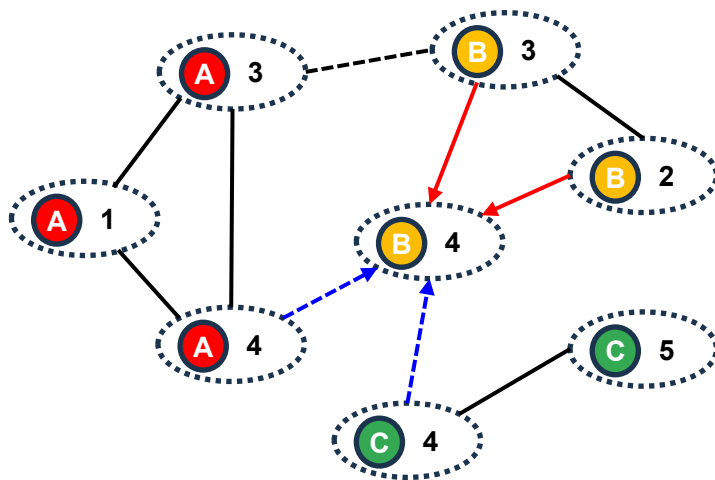
$$H^{(0)} = P_{vertex} X \in \mathbb{R}^{|\mathcal{V}_l| \times d_i}$$



□ Feature aggregation

- Convolve information from neighbors who share the same hyperedges (red)/vertices (blue)

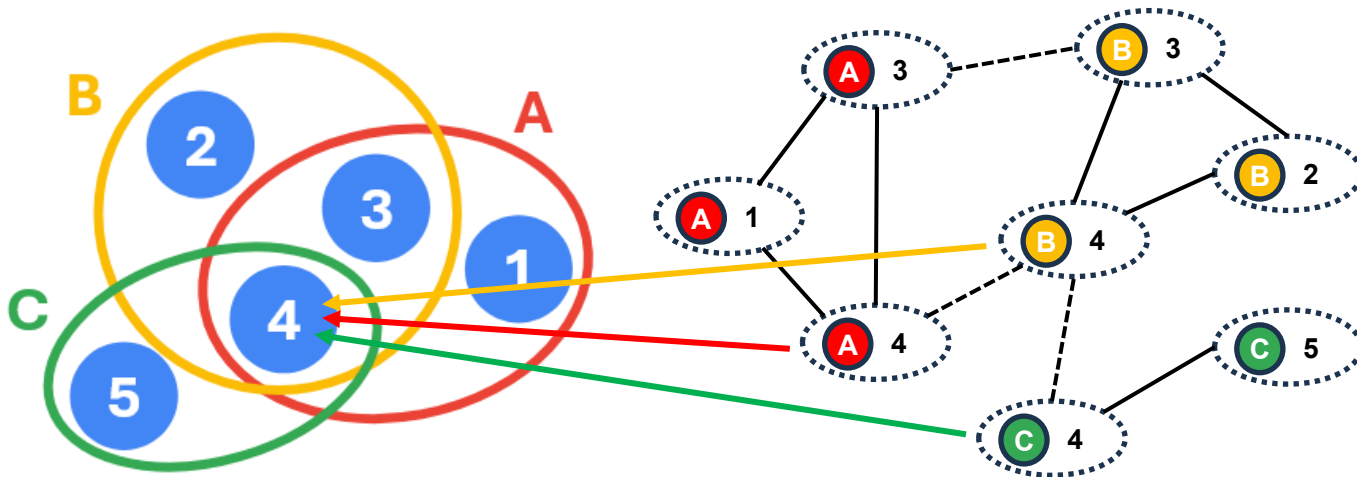
$$h_{(v,e)}^{(k+1)} = \sigma \left(\sum_{e'} w_e h_{(v,e')}^{(k)} \Theta^{(k)} + \sum_{v'} w_v h_{(v',e)}^{(k)} \Theta^{(k)} \right)$$



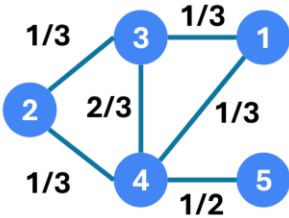
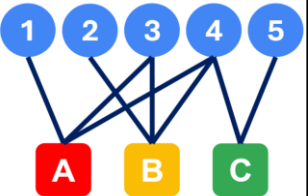
□ Representation Back-projection

- Fuse the learned representation using P'_{vertex} in an inverse edge degree manner

$$Y = P'_{vertex} H^{(K)} \in \mathbb{R}^{|\mathcal{V}| \times d_o}$$



Comparison of Expansion Schemes

Expansion	Graph Construction	Message passing	Pros	Cons
	Each hyperedge $e \rightarrow$ clique	Node-to-Node	Various GNN strategies can be directly applied	Higher-order information can be lost
	Add a hyperedge node for each hyperedge	Node-to-Hyperedge	Can maintain higher-order information	Direct GNN techniques can be non-trivial
	Each incidence (v, e) becomes a line node	Incidence-to-Incidence	Can maintain higher-order information	Memory Issue



Self-Supervised Hypergraph Learning

Problems

- Hypergraph data labeling is often **time, resource, and labor-intensive**



Problems

□ Real world (hyper)graphs are **sparse**

■ Most objects have only a few relationships

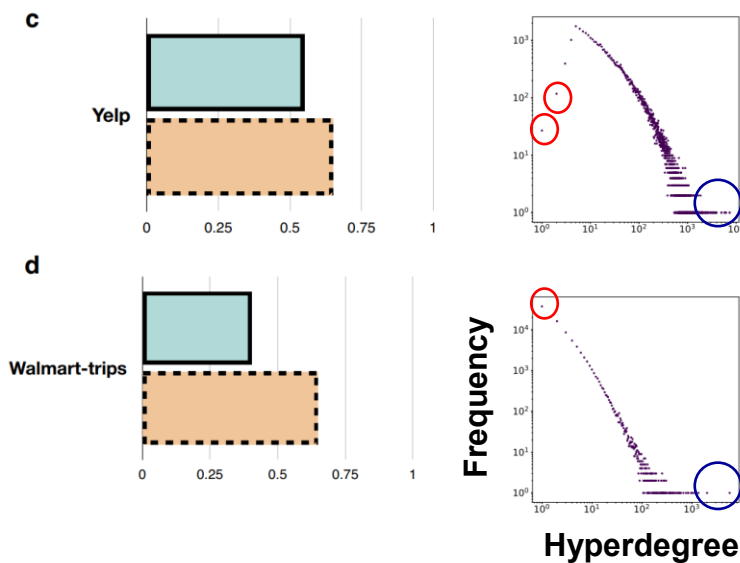
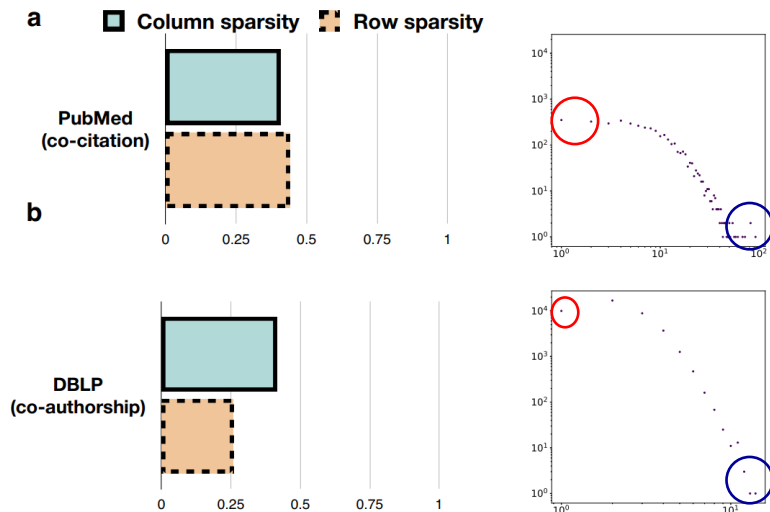
H

1			
1		1	
1	1	1	1
	1		1
		1	1

○ : Low-degree nodes

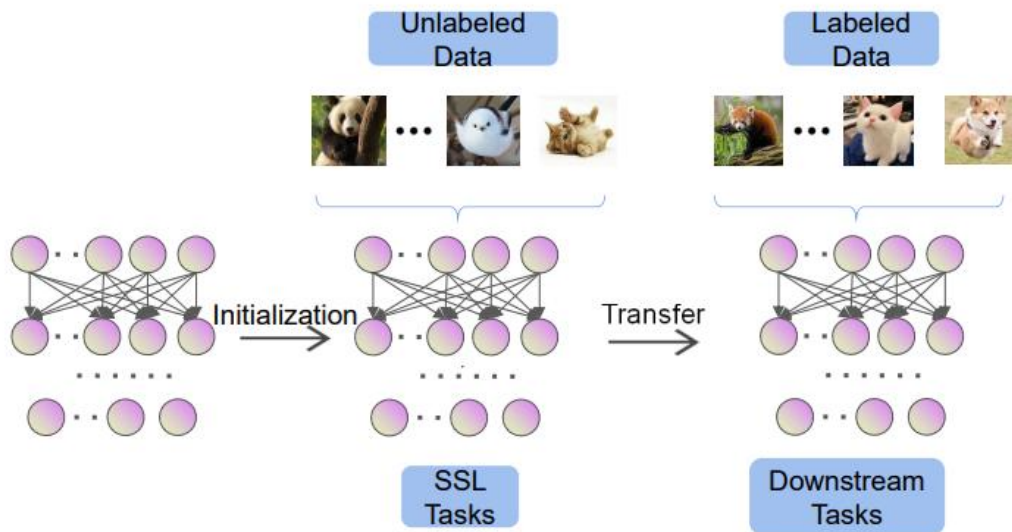
○ : High-degree nodes

Incidence matrix



□ Self-Supervised Learning (SSL)

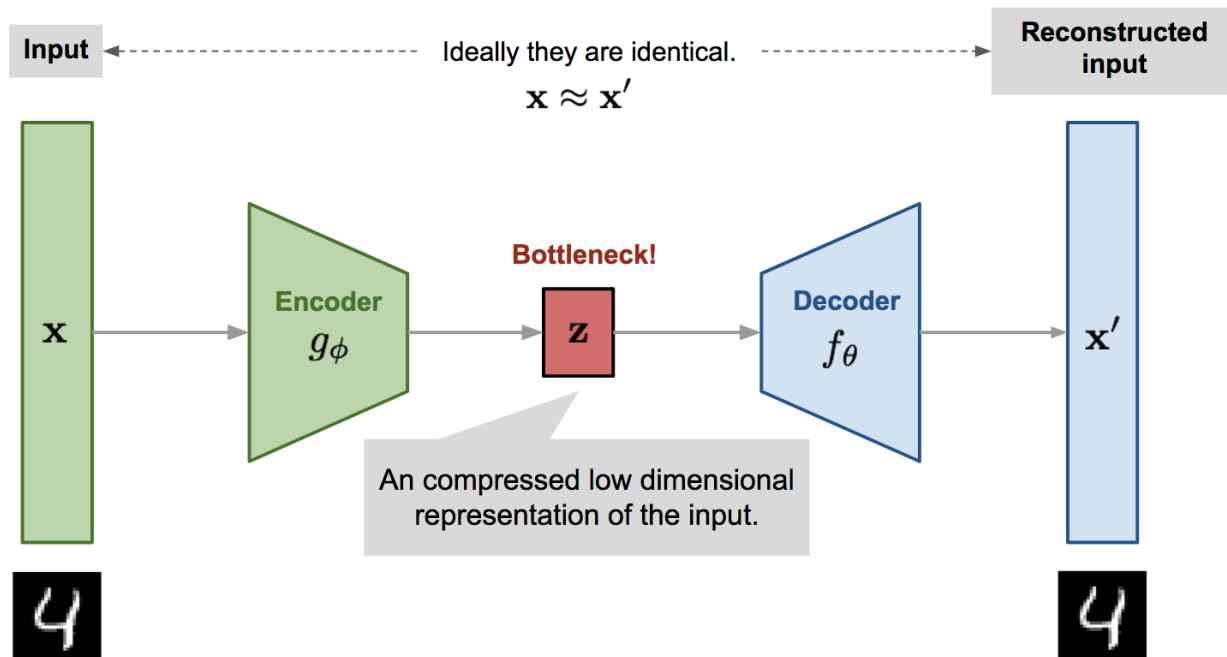
- Learn discriminative features from vast quantities of unlabeled instances without relying on human annotations



Generation-based Methods

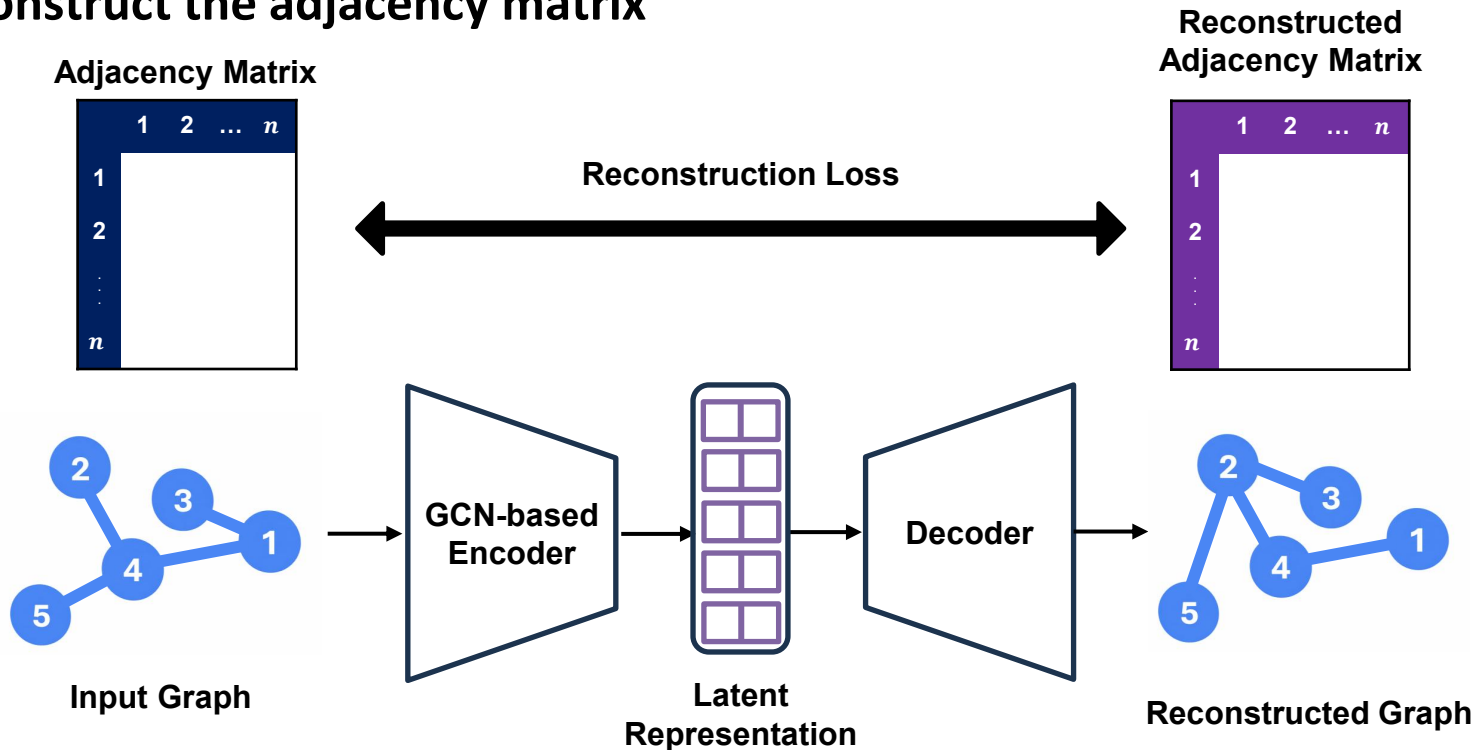
□ Key Idea

- Aim to **reconstruct the input data**



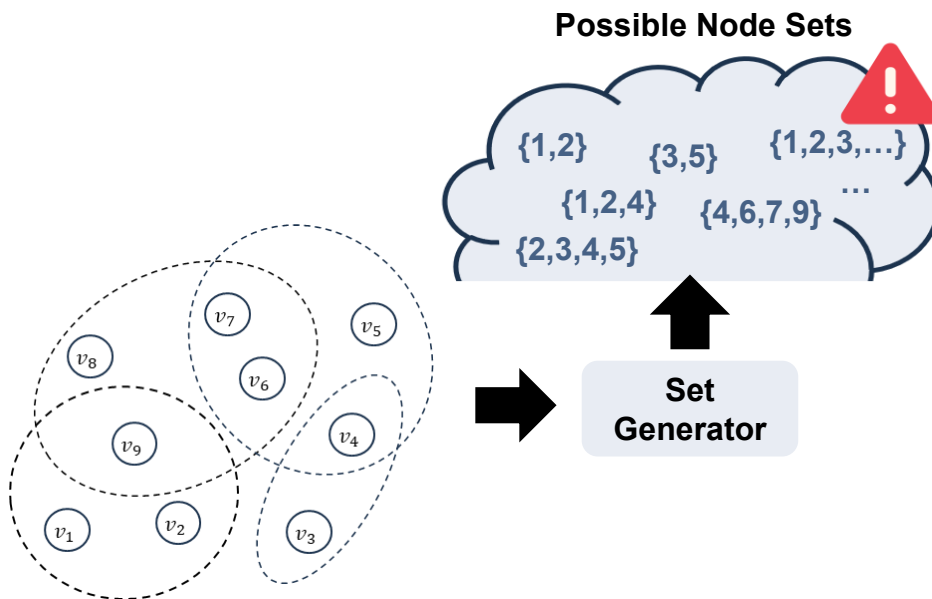
Generation-based Graph SSL

□ Reconstruct the adjacency matrix



□ How can we reconstruct incidence?

- Full incidence-matrix recovery is combinatorial and intractable





HypeBoy: Generative Self-Supervised Representation Learning on Hypergraphs

Sunwoo Kim, Shinhwan Kang, Fanchen Bu, Soo Yong Lee, Jaemin Yoo,
Kijung Shin

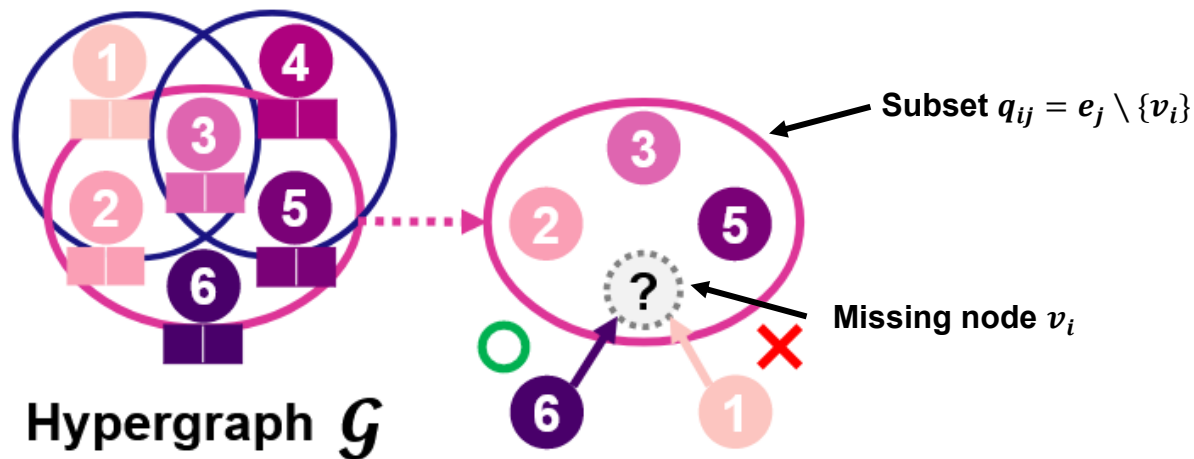
Korea Advanced Institute of Science and Technology (KAIST)

24-ICLR

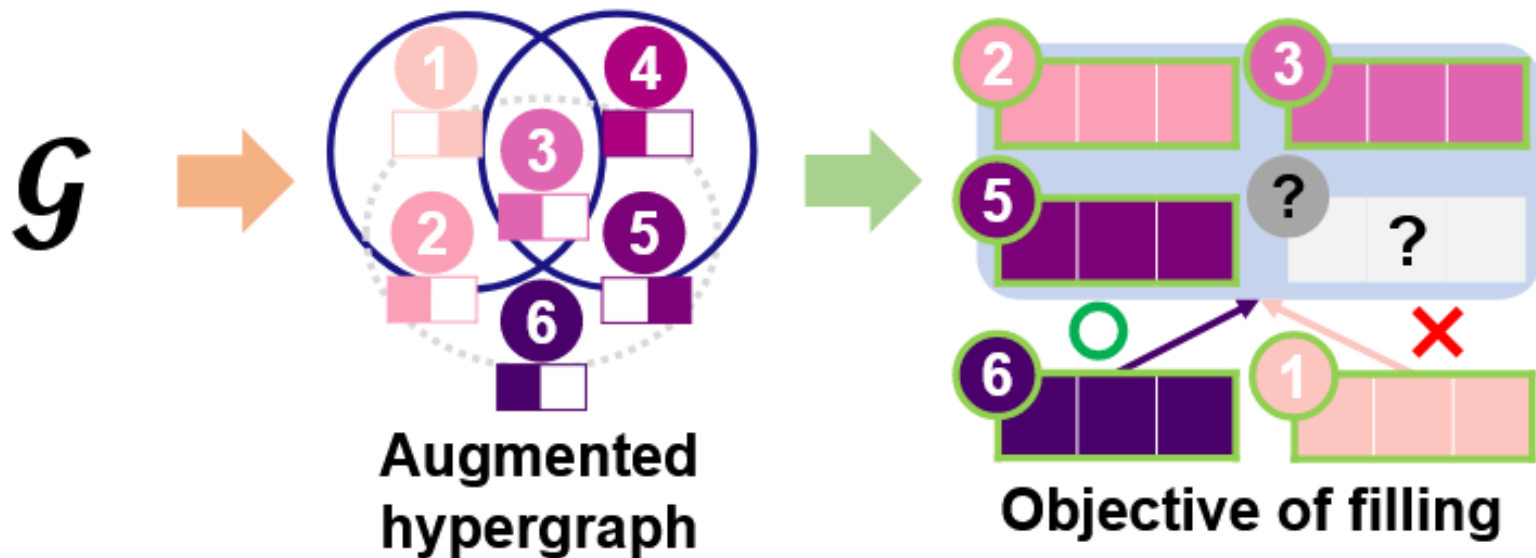
Hyperedge Filling

□ **Aim to predict a node that is likely to from a hyperedge with it**

■ Maximize the probability of v_i correctly completing q_{ij}



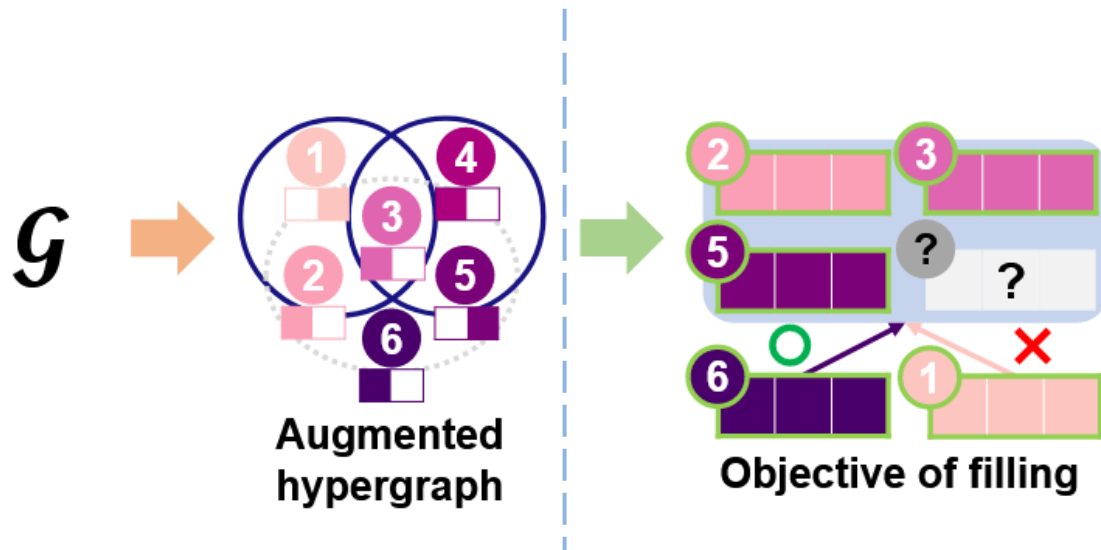
Hypeboy: Hypergraphs, build own hyperedges



Hypeboy: Hypergraphs, build own hyperedges

□ Hypergraph Augmentation

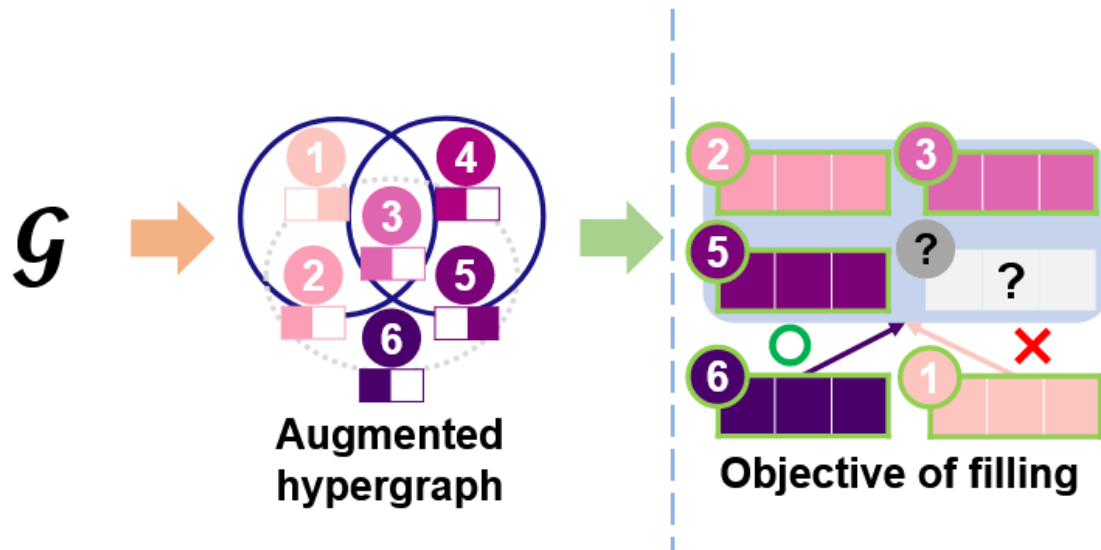
- Compose of feature masking and hyperedge sampling
- Allow the encoder to not overfit to the neighbor distribution and features



Hypeboy: Hypergraphs, build own hyperedges

□ Hypergraph Encoding

- Encode nodes and queries with an HNN encoder and projection heads
- Prevent dimensional collapse in node representations



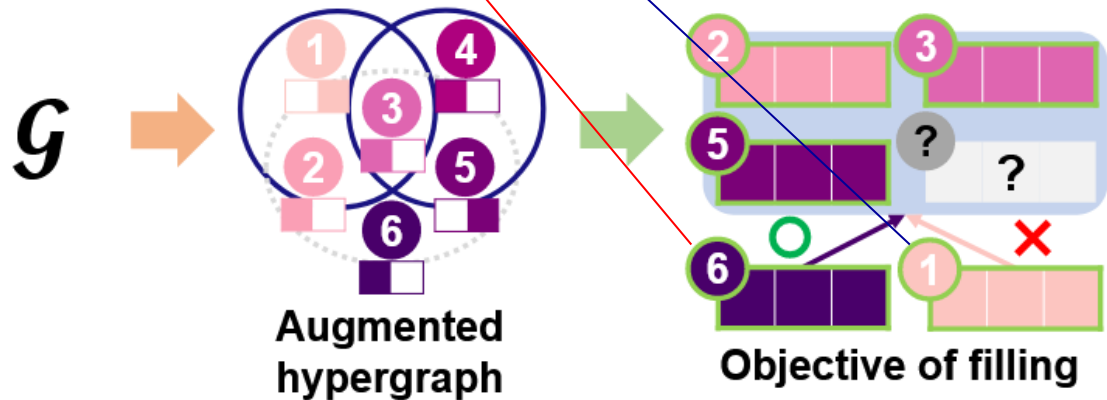
Hypeboy: Hypergraphs, build own hyperedges

□ Hyperedge Filling Loss

- Compute the SSL loss based on the hyperedge filling probability

$$\square p_{(\mathbf{x}, \varepsilon, \theta)}(v_i | q_{ij}) := \frac{\exp(\text{sim}(\mathbf{h}_i, \mathbf{q}_{ij}))}{\sum_{v_k \in \mathcal{V}} \exp(\text{sim}(\mathbf{h}_k, \mathbf{q}_{ij}))}$$

$$\square \mathcal{L} := -\sum_{e_j \in \mathcal{E}} \sum_{v_i \in e_j} \log p_{(\mathbf{x}, \varepsilon, \theta)}(v_i | q_{ij})$$



Hypeboy: Hypergraphs, build own hyperedges

□ Performance on Node Classification

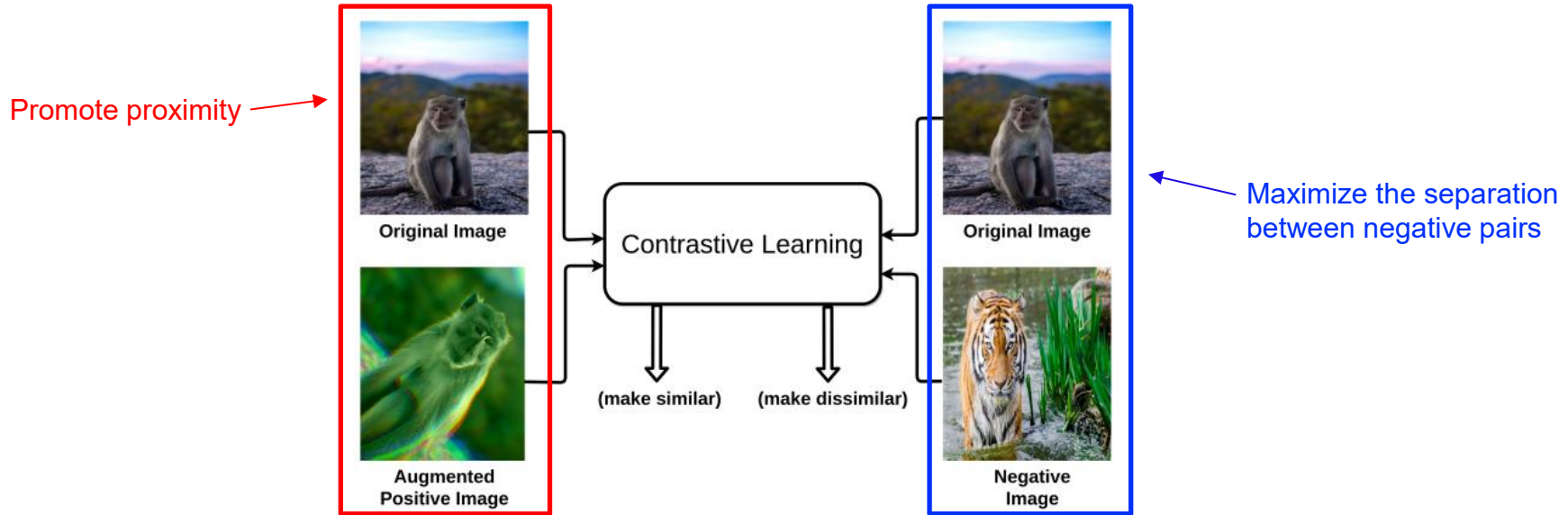
■ Show the best average ranking among all 17 methods

	Method	Citeseer	Cora	Pubmed	Cora-CA	DBLP-P	DBLP-A	AMiner	IMDB	MN-40	20News	House	A.R.
(Semi-)Supervised	R.G.	18.1 (0.9)	17.4 (1.0)	36.0 (0.7)	17.8 (0.7)	18.9 (0.2)	25.6 (1.0)	10.2 (0.2)	33.9 (0.7)	3.7 (0.2)	50.1 (1.3)	26.7 (0.3)	16.9
	MLP	32.5 (7.0)	27.9 (7.0)	62.1 (3.7)	34.8 (5.1)	73.5 (1.0)	56.0 (4.9)	22.3 (1.7)	39.1 (2.4)	89.4 (1.5)	73.1 (1.4)	72.2 (3.9)	13.4
	HGNN	41.9 (7.8)	50.0 (7.2)	72.9 (5.0)	50.2 (5.7)	85.3 (0.8)	67.1 (6.0)	30.3 (2.5)	42.2 (2.9)	88.0 (1.4)	76.4 (1.9)	52.7 (3.8)	10.0
	HyperGCN	31.4 (9.5)	33.1 (10.2)	63.5 (14.4)	37.1 (9.1)	53.5 (11.6)	68.2 (14.4)	26.4 (3.6)	37.9 (4.5)	55.1 (7.8)	67.0 (9.4)	49.8 (3.5)	14.7
	HNHN	43.1 (8.7)	50.0 (7.9)	72.1 (5.4)	48.3 (6.2)	84.6 (0.9)	62.6 (4.8)	30.0 (2.4)	42.3 (3.4)	86.1 (1.6)	74.2 (1.5)	49.7 (2.2)	11.9
	UniGCN	44.2 (8.1)	49.1 (8.4)	74.4 (3.9)	51.3 (6.3)	86.9 (0.6)	65.1 (4.7)	32.7 (1.8)	41.6 (3.5)	89.1 (1.0)	77.2 (1.2)	51.1 (2.4)	9.1
	UniGIN	40.4 (9.1)	47.8 (7.7)	69.8 (5.6)	48.3 (6.1)	83.4 (0.8)	63.4 (5.1)	30.2 (1.4)	41.4 (2.7)	88.2 (1.8)	70.6 (1.8)	51.1 (3.0)	13.0
	UniGCNII	44.2 (9.0)	48.5 (7.4)	74.1 (3.9)	54.8 (7.5)	87.4 (0.6)	65.8 (3.9)	32.5 (1.7)	42.5 (3.9)	90.8 (1.1)	70.9 (1.0)	50.8 (4.3)	8.8
	AllSet	43.5 (8.0)	47.6 (4.2)	72.4 (4.5)	57.5 (5.7)	85.9 (0.6)	65.3 (3.9)	29.3 (1.2)	42.3 (2.4)	92.1 (0.6)	71.9 (2.5)	54.1 (3.4)	9.8
	ED-HNN	40.3 (8.0)	47.6 (7.7)	72.7 (4.7)	54.8 (5.4)	86.2 (0.8)	65.8 (4.8)	30.0 (2.1)	41.4 (3.0)	90.7 (0.9)	76.2 (1.2)	71.3 (3.7)	9.6
Self-supervised	PhenomNN	49.8 (9.6)	56.4 (9.6)	76.1 (3.5)	60.8 (6.2)	88.1 (0.4)	72.3 (4.1)	33.8 (2.0)	44.1 (3.7)	95.9 (0.8)	74.0 (1.5)	70.4 (5.6)	3.9
	GraphMAE2	41.1 (10.0)	49.3 (8.3)	72.9 (4.2)	55.4 (8.4)	86.6 (0.6)	69.5 (4.4)	32.8 (1.9)	43.3 (2.7)	90.1 (0.7)	71.9 (1.3)	52.8 (3.5)	8.4
	MaskGAE	49.6 (10.1)	57.1 (8.8)	72.8 (4.3)	57.8 (5.9)	86.3 (0.5)	74.8 (3.1)	33.7 (1.6)	44.5 (2.5)	90.0 (0.9)	O.O.T.	51.8 (3.3)	7.4
	TriCL	51.7 (9.8)	60.2 (7.9)	76.2 (3.6)	64.3 (5.5)	88.0 (0.4)	79.7 (2.9)	33.1 (2.2)	46.9 (2.9)	90.3 (1.0)	77.2 (1.0)	69.7 (4.9)	3.4
	HyperGCL	47.0 (9.2)	60.3 (7.4)	76.8 (3.7)	62.0 (5.1)	87.6 (0.5)	79.7 (3.8)	33.2 (1.6)	43.9 (3.6)	91.2 (0.8)	77.8 (0.8)	69.2 (4.9)	3.5
	H-GD	45.4 (9.9)	50.6 (8.2)	74.5 (3.5)	58.8 (6.2)	87.3 (0.5)	75.1 (3.6)	32.6 (2.2)	43.0 (3.3)	90.0 (1.0)	77.2 (1.0)	69.7 (5.1)	5.8
	HYPEBOY	56.7 (9.8)	62.3 (7.7)	77.0 (3.4)	66.3 (4.6)	88.2 (0.4)	80.6 (2.3)	34.1 (2.2)	47.6 (2.5)	90.4 (0.9)	77.6 (0.9)	70.4 (4.8)	1.7

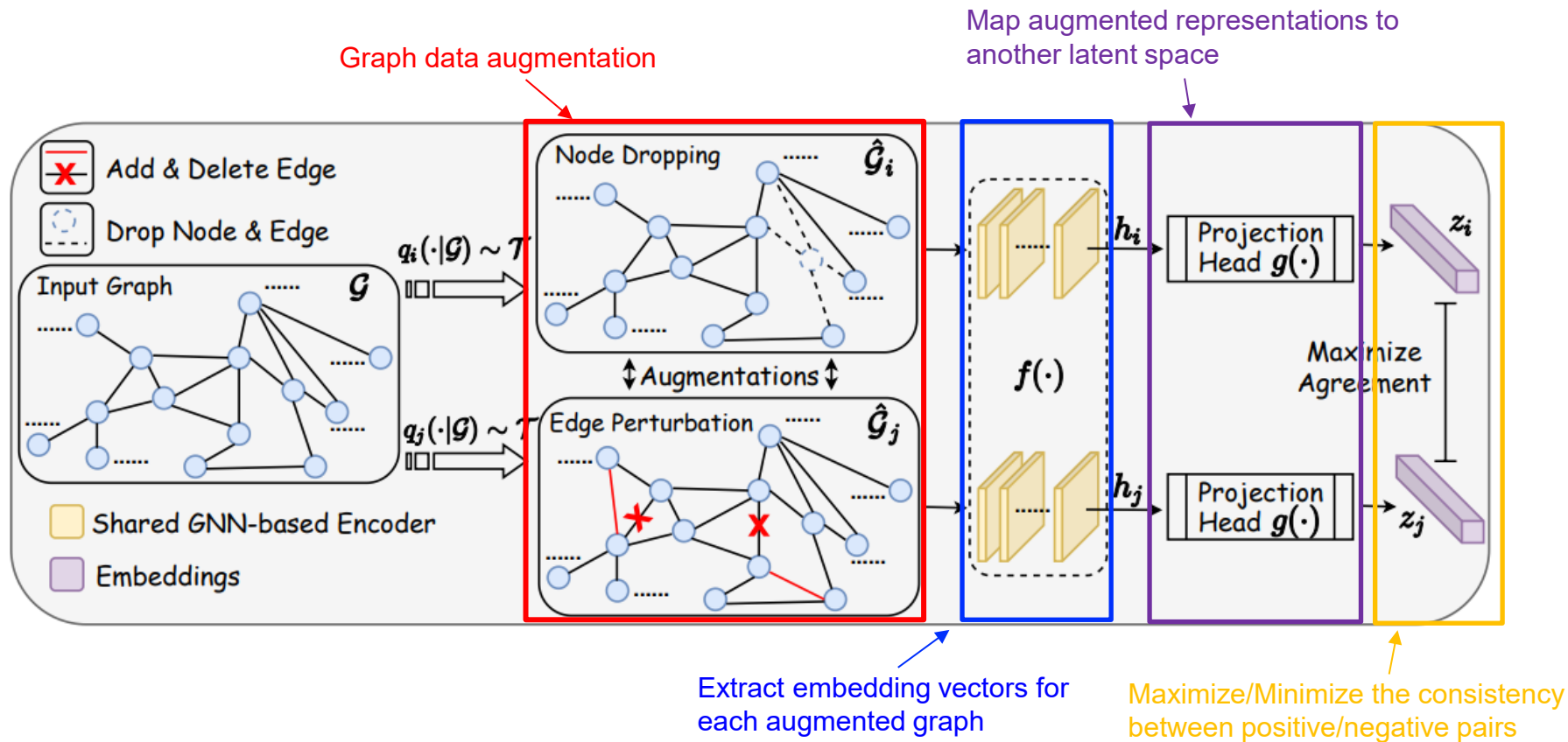
Contrast-based Methods

□ Key Idea

- **Push original and augmented data closer**, **push original and negative data away**



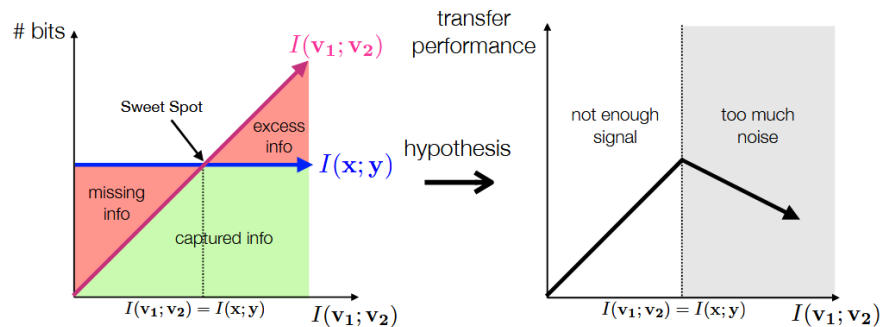
Contrast-based Graph SSL



Challenges

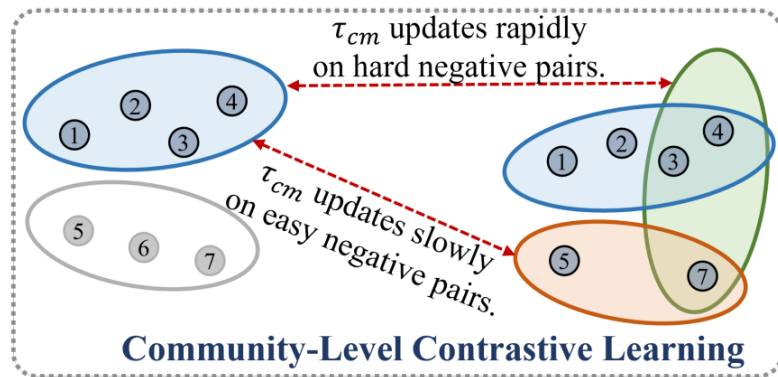
□ How to augment a hypergraph?

- The choice of views is what controls the information the representation captures



□ What to contrast?

- Node-only contrast cannot reflect higher-order information





I'm Me, We're Us, and I'm Us: Tri-directional Contrastive Learning on Hypergraphs

Dongjin Lee and Kijung Shin

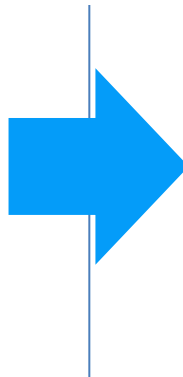
School of Electrical Engineering, Kim Jaechul Graduate School of AI, KAIST, South Korea

23-AAAI

□ Motivation

Node-only Contrast

- Cannot capture higher-order relations
- Lead to limited expressiveness



Tri-directional Contrast

- Node + Group + Membership contrast
- Preserve higher-order structural information
- Produce richer, more generalizable embeddings

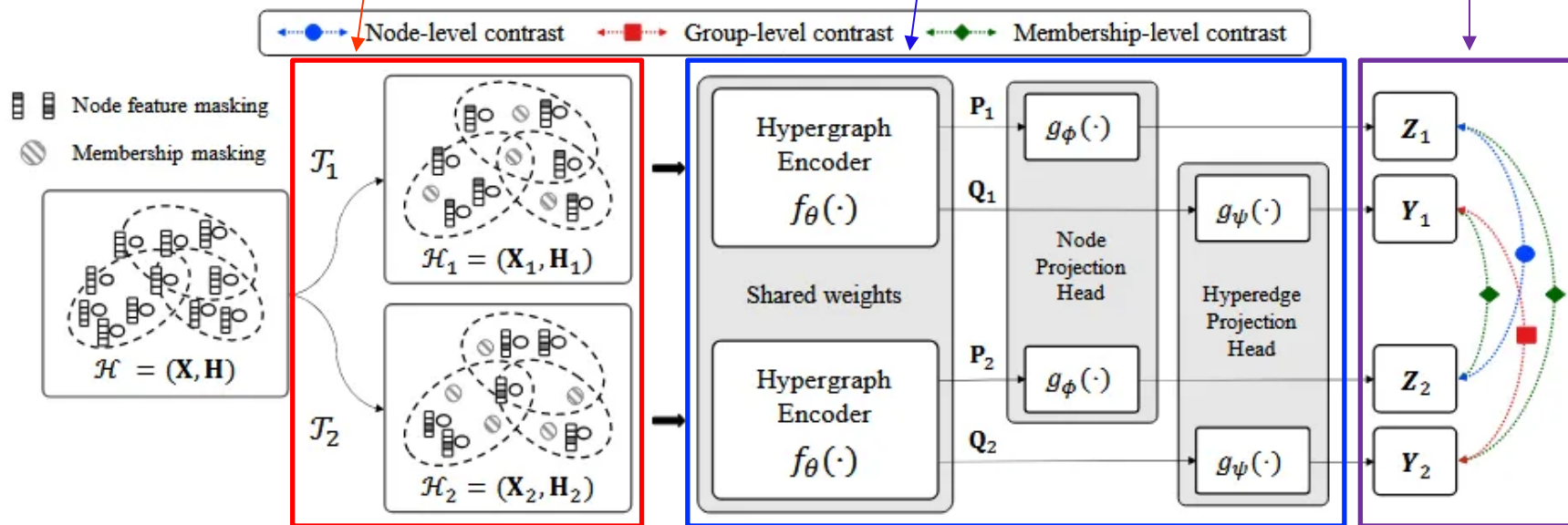
How to augment a hypergraph?	What to contrast?
Δ	$\bigcirc$

Overview

Generalized Hyperedge
Perturbation + Attribute masking

Form node and hyperedge
representations

Apply three forms
of contrast

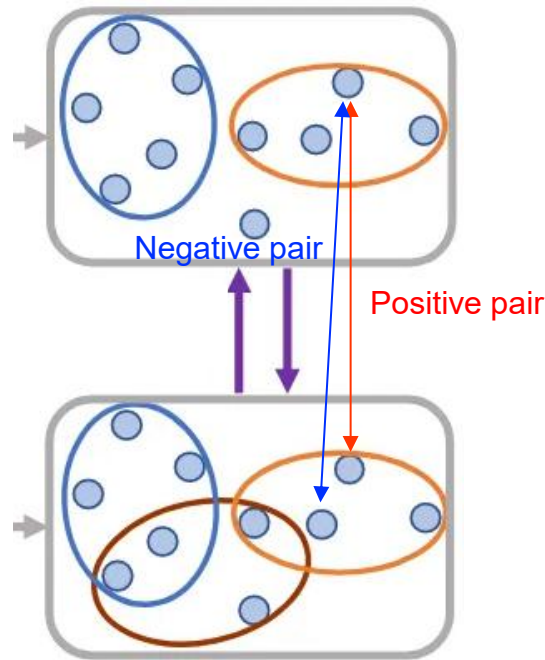


□ Node-level contrast

- Discriminate the representations of the same node in the two augmented views from other node representations

$$\ell_n(\mathbf{z}_{1,i}, \mathbf{z}_{2,i}) = -\log \frac{e^{s(\mathbf{z}_{1,i}, \mathbf{z}_{2,i})/\tau_n}}{\sum_{k=1}^{|V|} e^{s(\mathbf{z}_{1,i}, \mathbf{z}_{2,k})/\tau_n}},$$

$$\mathcal{L}_n = \frac{1}{2|V|} \sum_{i=1}^{|V|} \{\ell_n(\mathbf{z}_{1,i}, \mathbf{z}_{2,i}) + \ell_n(\mathbf{z}_{2,i}, \mathbf{z}_{1,i})\}.$$

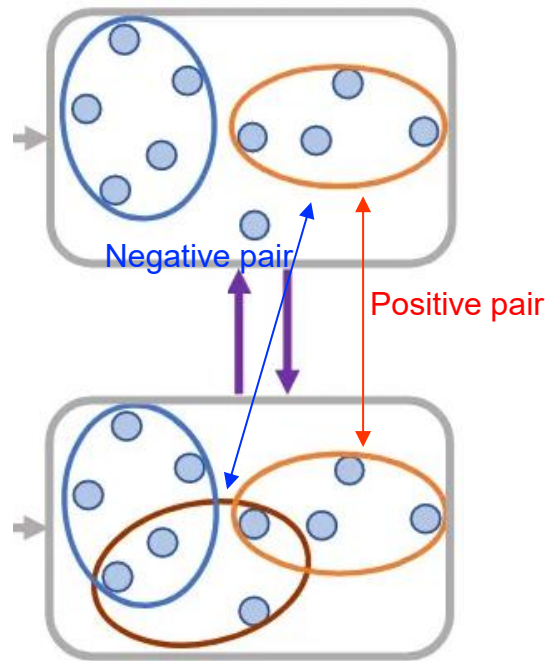


□ Group-level contrast

- Distinguish the representations of the same hyperedge in the two augmented views from other hyperedge representations

$$\ell_g(\mathbf{y}_{1,j}, \mathbf{y}_{2,j}) = -\log \frac{e^{s(\mathbf{y}_{1,j}, \mathbf{y}_{2,j})/\tau_g}}{\sum_{k=1}^{|E|} e^{s(\mathbf{y}_{1,j}, \mathbf{y}_{2,k})/\tau_g}},$$

$$\mathcal{L}_g = \frac{1}{2|E|} \sum_{j=1}^{|E|} \{\ell_g(\mathbf{y}_{1,j}, \mathbf{y}_{2,j}) + \ell_g(\mathbf{y}_{2,j}, \mathbf{y}_{1,j})\}.$$

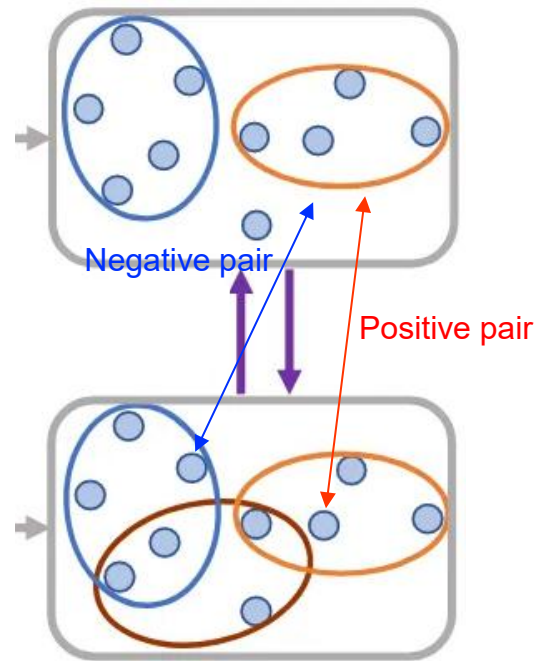


□ Membership-level contrast

- Learn to represent if relationships actually exist and to move away if they don't exist

$$\ell_m(\mathbf{z}_i, \mathbf{y}_j) = -\log \underbrace{\frac{e^{\mathcal{D}(\mathbf{z}_i, \mathbf{y}_j)/\tau_m}}{e^{\mathcal{D}(\mathbf{z}_i, \mathbf{y}_j)/\tau_m} + \sum_{k:i \notin k} e^{\mathcal{D}(\mathbf{z}_i, \mathbf{y}_k)/\tau_m}}}_{\text{when } \mathbf{z}_i \text{ is the anchor}} - \log \underbrace{\frac{e^{\mathcal{D}(\mathbf{z}_i, \mathbf{y}_j)/\tau_m}}{e^{\mathcal{D}(\mathbf{z}_i, \mathbf{y}_j)/\tau_m} + \sum_{k:k \notin j} e^{\mathcal{D}(\mathbf{z}_k, \mathbf{y}_j)/\tau_m}}}_{\text{when } \mathbf{y}_j \text{ is the anchor}},$$

$$\mathcal{L}_m = \frac{1}{2K} \sum_{i=1}^{|V|} \sum_{j=1}^{|E|} \mathbb{1}_{[h_{ij}=1]} \{ \ell_m(\mathbf{z}_{1,i}, \mathbf{y}_{2,j}) + \ell_m(\mathbf{z}_{2,i}, \mathbf{y}_{1,j}) \}.$$



□ Performance on Node Classification

- Graph contrastive learning methods show significantly lower accuracy compared to TriCL

Table 1: Node classification accuracy and standard deviations. Graph methods, marked as $\star$, are applied after converting hypergraphs to graphs via clique expansion. For each dataset, the best and the second-best performances are highlighted in **boldface** and underlined, respectively. A.R. denotes average rank, OOT denotes cases where results are not obtained within 24 hours, and OOM indicates out of memory on a 24GB GPU. In most cases, TriCL outperforms all others, including the supervised ones.

	Method	Cora-C	Citeseer	Pubmed	Cora-A	DBLP	Zoo	20News	Mushroom	NTU2012	ModelNet40	A.R.↓
Supervised	MLP	60.32 $\pm$ 1.5	62.06 $\pm$ 2.3	76.27 $\pm$ 1.1	64.05 $\pm$ 1.4	81.18 $\pm$ 0.2	75.62 $\pm$ 9.5	79.19 $\pm$ 0.5	99.58 $\pm$ 0.3	65.17 $\pm$ 2.3	93.75 $\pm$ 0.6	12.5
	GCN $\star$	77.11 $\pm$ 1.8	66.07 $\pm$ 2.4	82.63 $\pm$ 0.6	73.66 $\pm$ 1.3	87.58 $\pm$ 0.2	36.79 $\pm$ 9.6	OOM	92.47 $\pm$ 0.9	71.17 $\pm$ 2.4	91.67 $\pm$ 0.2	11.7
	GAT $\star$	77.75 $\pm$ 2.1	67.62 $\pm$ 2.5	81.96 $\pm$ 0.7	74.52 $\pm$ 1.3	88.59 $\pm$ 0.1	36.48 $\pm$ 10.0	OOM	OOM	70.94 $\pm$ 2.6	91.43 $\pm$ 0.3	11
	HGNN	77.50 $\pm$ 1.8	66.16 $\pm$ 2.3	83.52 $\pm$ 0.7	74.38 $\pm$ 1.2	88.32 $\pm$ 0.3	78.58 $\pm$ 11.1	80.15 $\pm$ 0.3	98.59 $\pm$ 0.5	72.03 $\pm$ 2.4	92.23 $\pm$ 0.2	8.1
	HyperConv	76.19 $\pm$ 2.1	64.12 $\pm$ 2.6	83.42 $\pm$ 0.6	73.52 $\pm$ 1.0	88.83 $\pm$ 0.2	62.53 $\pm$ 14.5	79.83 $\pm$ 0.4	97.56 $\pm$ 0.6	72.62 $\pm$ 2.6	91.84 $\pm$ 0.1	9.8
	HNHN	76.21 $\pm$ 1.7	67.28 $\pm$ 2.2	80.97 $\pm$ 0.9	74.88 $\pm$ 1.6	86.71 $\pm$ 1.2	78.89 $\pm$ 10.2	79.51 $\pm$ 0.4	99.78 $\pm$ 0.1	71.45 $\pm$ 3.2	92.96 $\pm$ 0.2	8.9
	HyperGCN	64.11 $\pm$ 7.4	59.92 $\pm$ 9.6	78.40 $\pm$ 9.2	60.65 $\pm$ 9.2	76.59 $\pm$ 7.6	40.86 $\pm$ 2.1	77.31 $\pm$ 6.0	48.26 $\pm$ 0.3	46.05 $\pm$ 3.9	69.23 $\pm$ 2.8	15.1
	HyperSAGE	64.98 $\pm$ 5.3	52.43 $\pm$ 9.4	79.49 $\pm$ 8.7	64.59 $\pm$ 4.3	79.63 $\pm$ 8.6	40.86 $\pm$ 2.1	OOT	OOT	OOT	OOT	14.7
	UniGCN	77.91 $\pm$ 1.9	66.40 $\pm$ 1.9	84.08 $\pm$ 0.7	77.30 $\pm$ 1.4	90.31 $\pm$ 0.2	72.10 $\pm$ 12.1	80.24 $\pm$ 0.4	98.84 $\pm$ 0.5	73.27 $\pm$ 2.7	94.62 $\pm$ 0.2	5.9
	AllSet	76.21 $\pm$ 1.7	67.83 $\pm$ 1.8	82.85 $\pm$ 0.9	76.94 $\pm$ 1.3	90.07 $\pm$ 0.3	72.72 $\pm$ 11.8	79.90 $\pm$ 0.4	99.78 $\pm$ 0.1	75.09 $\pm$ 2.5	96.85 $\pm$ 0.2	6.2
Unsupervised	Node2vec $\star$	70.99 $\pm$ 1.4	53.85 $\pm$ 1.9	78.75 $\pm$ 0.9	58.50 $\pm$ 2.1	72.09 $\pm$ 0.3	17.02 $\pm$ 4.1	63.35 $\pm$ 1.7	88.16 $\pm$ 0.8	67.72 $\pm$ 2.1	84.94 $\pm$ 0.4	15.6
	DGI $\star$	78.17 $\pm$ 1.4	68.81 $\pm$ 1.8	80.83 $\pm$ 0.6	76.94 $\pm$ 1.1	88.00 $\pm$ 0.2	36.54 $\pm$ 9.7	OOM	OOM	72.01 $\pm$ 2.5	92.18 $\pm$ 0.2	9.3
	GRACE $\star$	79.11 $\pm$ 1.7	68.65 $\pm$ 1.7	80.08 $\pm$ 0.7	76.59 $\pm$ 1.0	OOM	37.07 $\pm$ 9.3	OOM	OOM	70.51 $\pm$ 2.4	90.68 $\pm$ 0.3	10.4
	S ² -HHGR	78.08 $\pm$ 1.7	68.21 $\pm$ 1.8	82.13 $\pm$ 0.6	78.15 $\pm$ 1.1	88.69 $\pm$ 0.2	80.06 $\pm$ 11.1	79.75 $\pm$ 0.3	97.15 $\pm$ 0.5	73.95 $\pm$ 2.4	93.26 $\pm$ 0.2	6.8
	Random-Init	63.62 $\pm$ 3.1	60.44 $\pm$ 2.5	67.49 $\pm$ 2.2	66.27 $\pm$ 2.2	76.57 $\pm$ 0.6	78.43 $\pm$ 11.0	77.14 $\pm$ 0.6	97.40 $\pm$ 0.6	74.39 $\pm$ 2.6	96.29 $\pm$ 0.3	11.9
	TriCL-N	80.23 $\pm$ 1.2	70.28 $\pm$ 1.5	83.44 $\pm$ 0.6	81.94 $\pm$ 1.1	90.88 $\pm$ 0.1	79.94 $\pm$ 11.1	<u>80.18 $\pm$ 0.2</u>	99.76 $\pm$ 0.2	75.20 $\pm$ 2.6	97.01 $\pm$ 0.2	3.4
	TriCL-NG	81.45 $\pm$ 1.2	71.38 $\pm$ 1.2	83.68 $\pm$ 0.7	82.00 $\pm$ 1.0	90.94 $\pm$ 0.1	80.19 $\pm$ 11.1	<u>80.18 $\pm$ 0.2</u>	99.81 $\pm$ 0.1	75.25 $\pm$ 2.5	97.02 $\pm$ 0.1	2
	TriCL	81.57 $\pm$ 1.1	72.02 $\pm$ 1.2	84.26 $\pm$ 0.6	82.15 $\pm$ 0.9	91.12 $\pm$ 0.1	80.25 $\pm$ 11.2	80.14 $\pm$ 0.2	99.83 $\pm$ 0.1	<u>75.23 $\pm$ 2.4</u>	97.08 $\pm$ 0.1	1.5

□ Ablation Study

- Considering the different types of contrast together can help improve performance

Table 2: Comparison of node classification accuracy according to whether or not to use each type of contrast (i.e., $\mathcal{L}_n$, $\mathcal{L}_g$, and $\mathcal{L}_m$). Using all types of contrasts (i.e., node-, group-, and membership-level contrast) achieves the best performance in most cases as they are complementarily reinforcing each other.

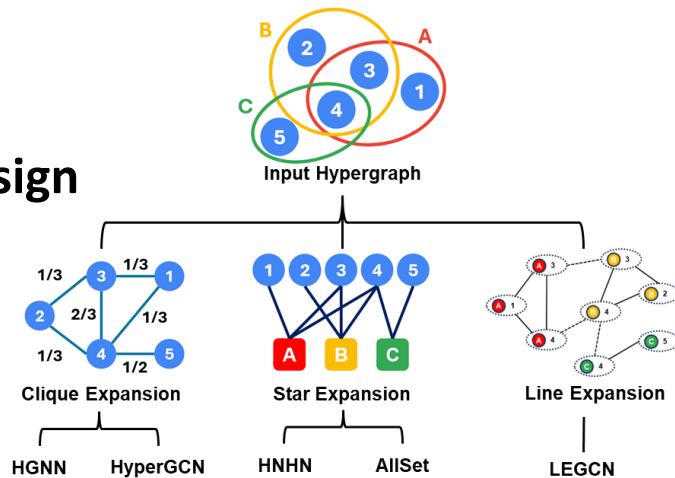
$\mathcal{L}_n$	$\mathcal{L}_g$	$\mathcal{L}_m$	Cora-C	Citeseer	Pubmed	Cora-A	DBLP	Zoo	20News	Mushroom	NTU2012	ModelNet40	A.R.↓
✓	-	-	80.23 ± 1.2	70.28 ± 1.5*	83.44 ± 0.6	81.94 ± 1.1	90.88 ± 0.1	79.94 ± 11.1	80.18 ± 0.2	99.76 ± 0.2	75.20 ± 2.6	97.01 ± 0.2	3.8
-	✓	-	79.69 ± 1.6	71.02 ± 1.3*	80.20 ± 1.3	78.98 ± 1.4	88.60 ± 0.2	79.31 ± 10.7	79.35 ± 0.4	99.13 ± 0.3	74.41 ± 2.6	96.66 ± 0.2	5.7
-	-	✓	76.76 ± 1.8	63.98 ± 2.0	79.86 ± 0.9	76.77 ± 1.1	63.95 ± 7.2	79.80 ± 11.0	79.27 ± 0.3	94.87 ± 0.7	73.11 ± 2.8	96.57 ± 0.2	6.9
✓	✓	-	81.45 ± 1.2	71.38 ± 1.4	83.68 ± 0.7	82.00 ± 1.0	90.94 ± 0.1	80.19 ± 11.1	80.18 ± 0.2	99.81 ± 0.1	75.25 ± 2.5	97.02 ± 0.1	<u>2.3</u>
✓	-	✓	80.49 ± 1.3	70.46 ± 1.5	83.98 ± 0.7	81.62 ± 1.0	90.75 ± 0.1	80.19 ± 11.1	80.15 ± 0.2	99.74 ± 0.2	75.12 ± 2.5	97.03 ± 0.1	3.6
-	✓	✓	80.80 ± 1.1	71.73 ± 1.4	82.81 ± 0.7	80.24 ± 1.0	90.17 ± 0.1	80.20 ± 11.1	79.29 ± 0.2	99.82 ± 0.1	73.76 ± 2.5	96.74 ± 0.1	4.1
✓	✓	✓	81.57 ± 1.1	72.02 ± 1.4	84.26 ± 0.6	82.15 ± 0.9	91.12 ± 0.1	80.25 ± 11.2	80.14 ± 0.2	99.83 ± 0.1	<u>75.23 ± 2.4</u>	97.08 ± 0.1	1.4

Conclusion

□ Hypergraphs Model Higher-Order Interactions

□ Hypergraph Expansions for Message Passing Design

- Clique Expansion: Vertex-to-Vertex
- Star Expansion: Vertex-to-Hyperedge
- Line Expansion: Incidence-to-Incidence



□ Self-Supervised Hypergraph Learning

- Generation-based Methods
- Contrast-based Methods

